Lab 3

Working with Git History, Stashing, and Reverting Commits

DevOps F26

Name	Sidra Liaqat
Roll number	2023-BCS-083
Section	A
GitHub repository link	https://github.com/sidraliaqat/DevOps

Submission details

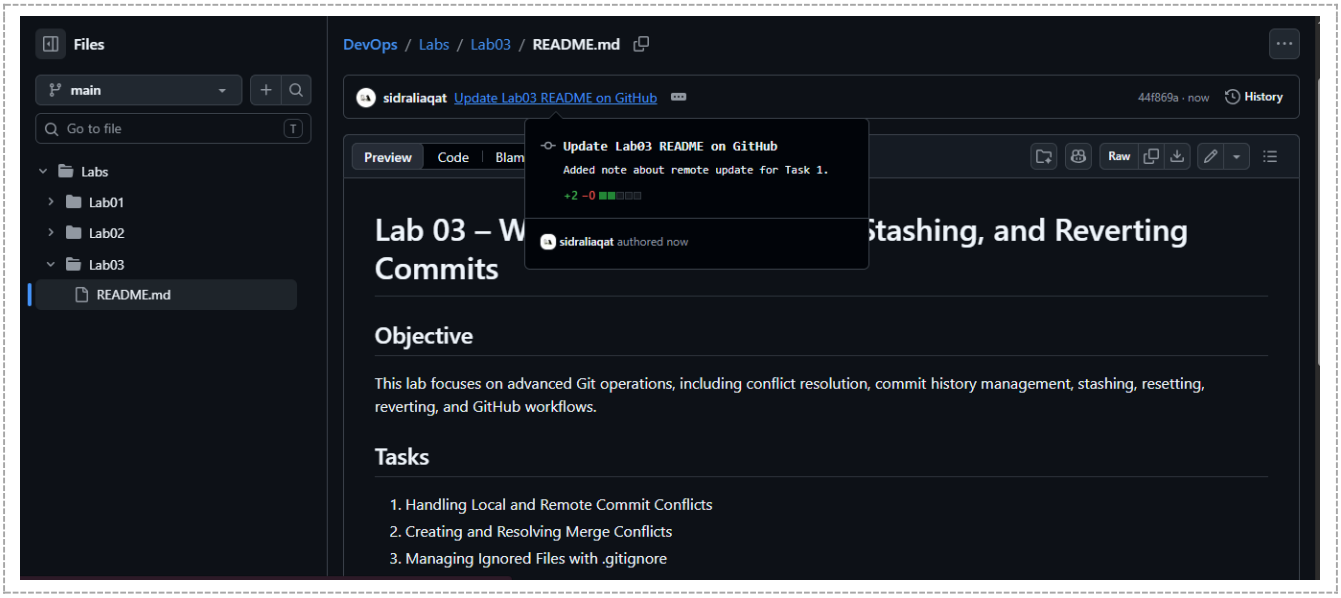
Repository: DevOps
Directory: Labs/Lab03
Screenshots: Labs/Lab03/screenshots

Task 1: Handling Local and Remote Commit Conflicts (Pull vs Pull --rebase)

Both the local and remote repositories get new commits, and the push is fixed first with a merge pull and then with a rebase pull.

1. On GitHub, edit README.md in the browser and add the line "This line was added remotely from GitHub." Commit it with the message "Remote update to README".

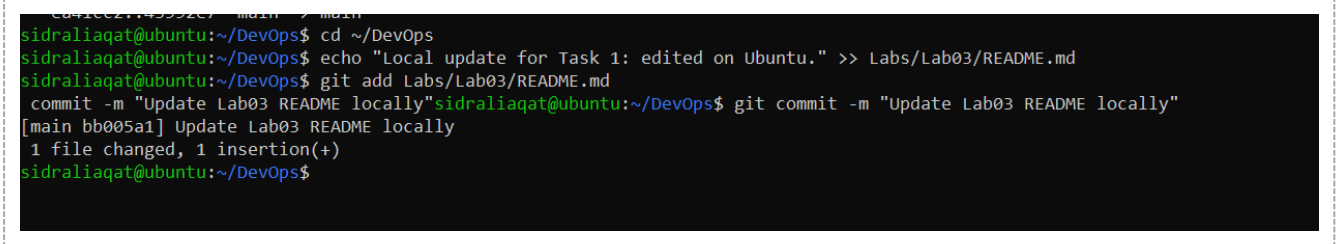
   Screenshot: remote_edit.png



2. Locally, edit README.md and add "This line was added locally." Then stage and commit.

```
git add README.md
git commit -m "Local update to README"
```

   Screenshot: local_commit.png



3. Try to push. The remote rejects it.

```
git push origin main
```

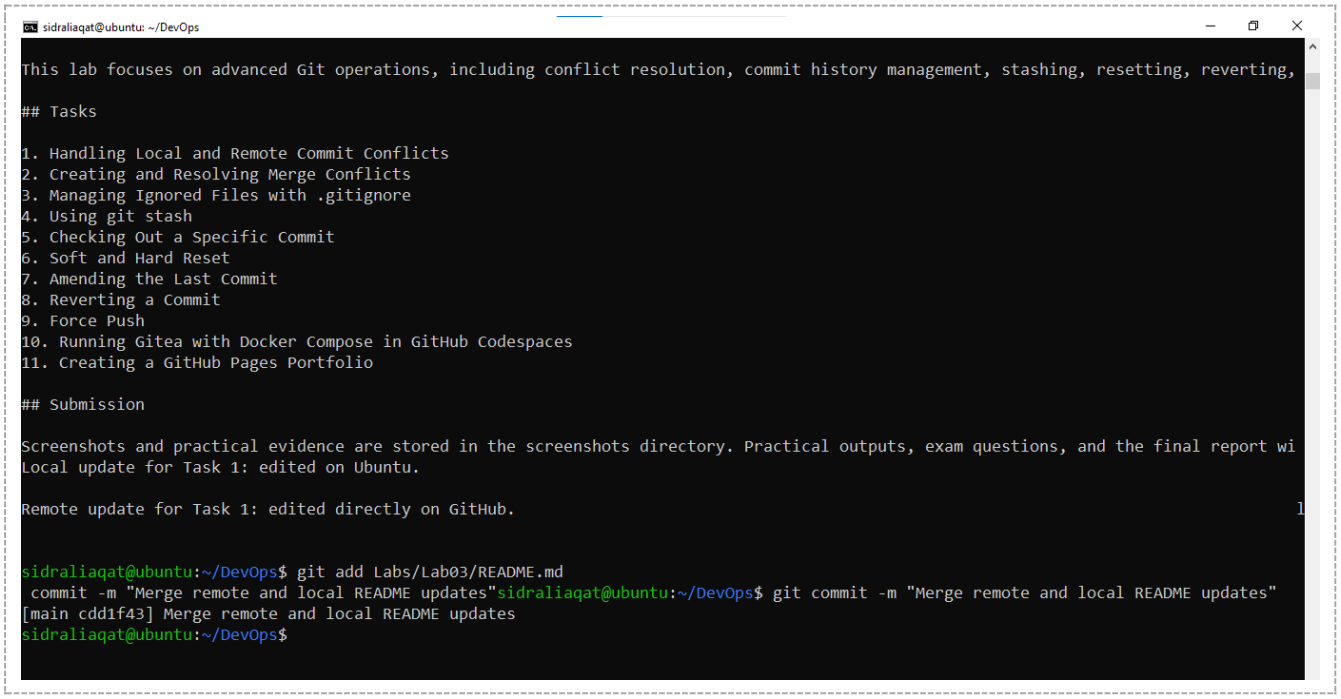
Screenshot: push_error.png



4. Pull with a merge, which creates a merge commit.

```
git pull --no-rebase origin main
```

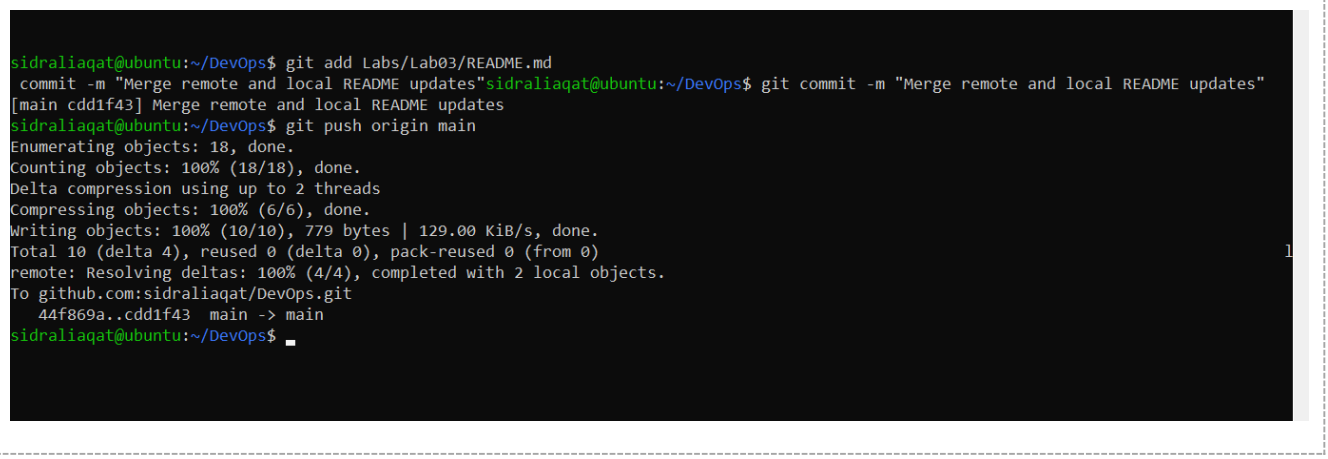
Screenshot: merge_commit.png



5. Push again.

```
git push -u origin main
```

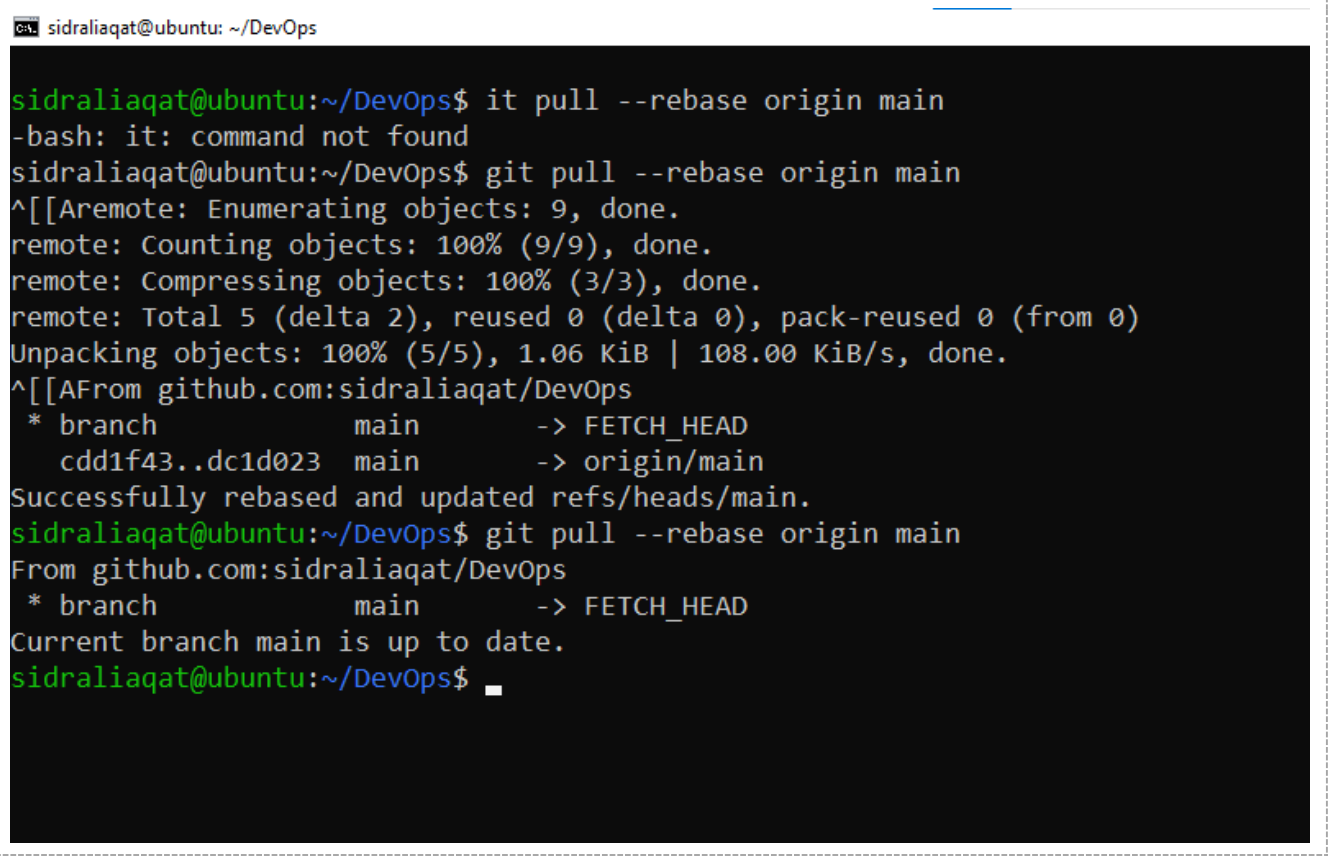
Screenshot: push_after_merge.png



6. Make another remote edit on GitHub and another local commit, then pull with rebase.

```
git pull --rebase origin main
```

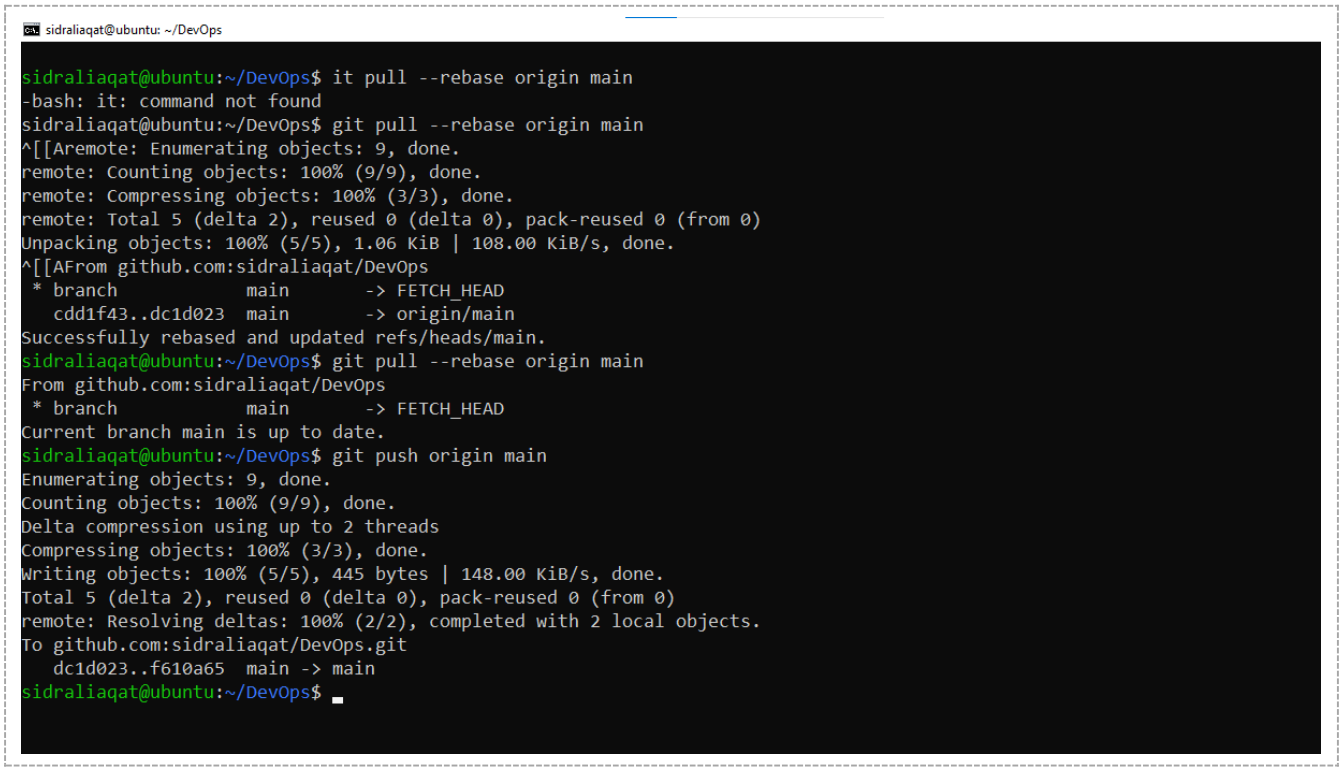
Screenshot: rebase_pull.png



7. Push again.

```
git push -u origin main
```

Screenshot: push_after_rebase.png

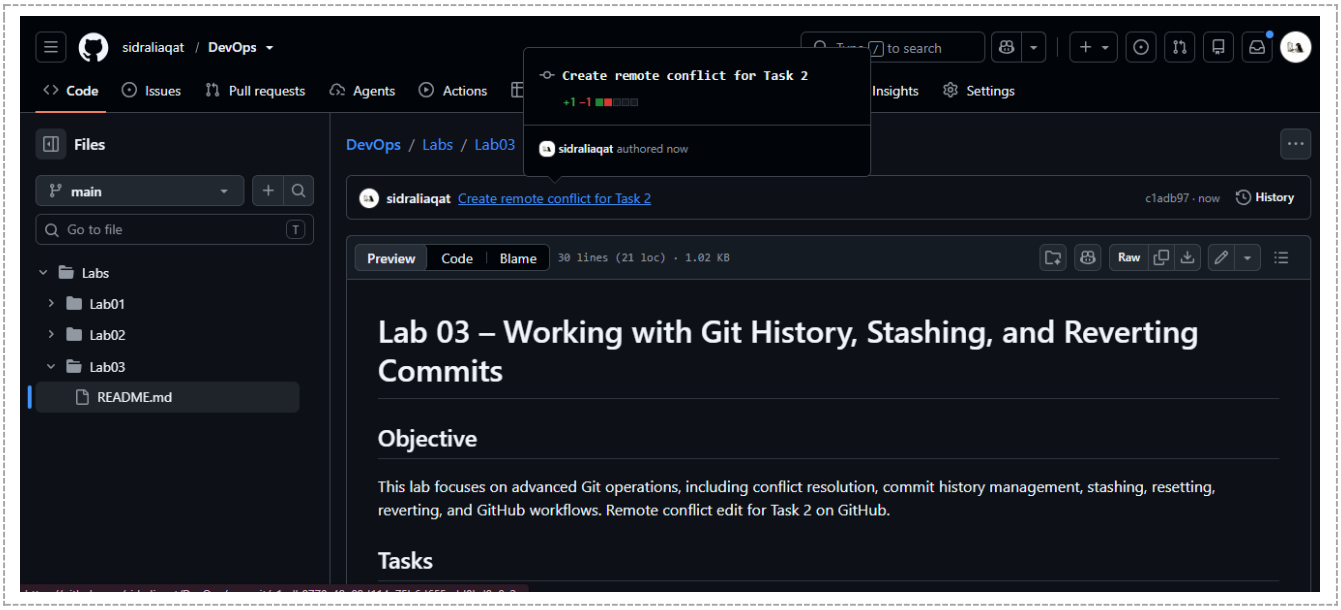


Task 2: Creating and Resolving Merge Conflicts Manually

Two different edits to the same line force Git to stop the rebase until the conflict is fixed by hand.

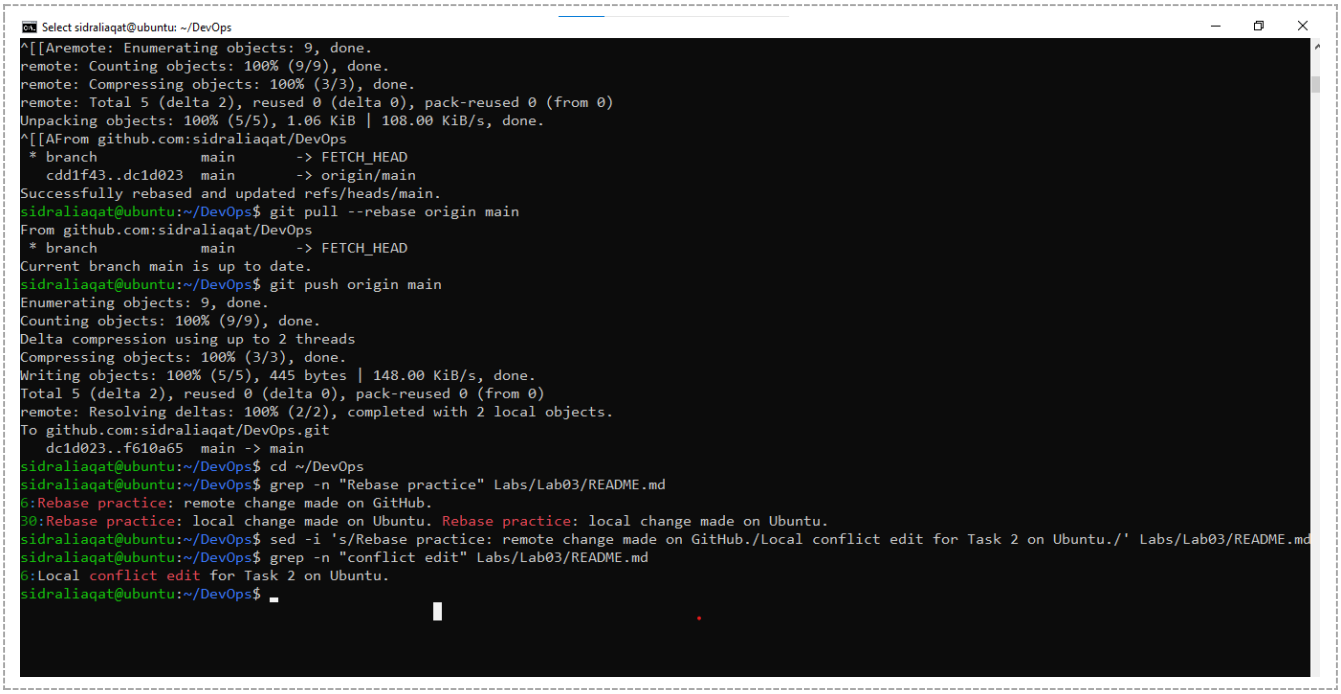
1. On GitHub, change the line to "This line was updated remotely again." Commit with the message "Remote conflicting change".

   Screenshot: remote_conflict_edit.png



2. Locally, change the same line to "This line was updated locally at the same time."

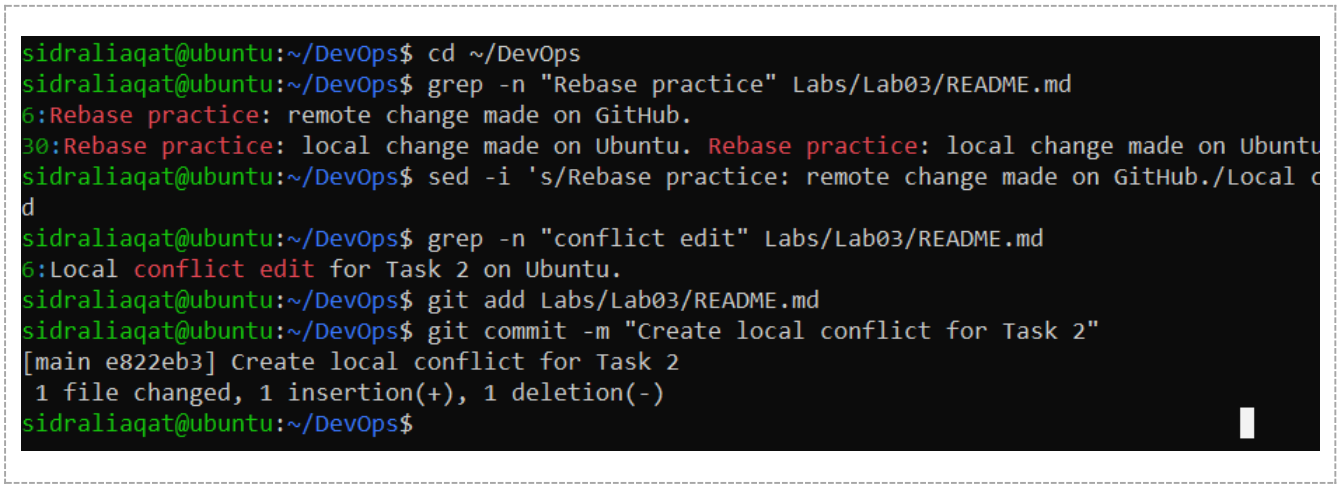
   Screenshot: local_conflict_edit.png



3. Stage and commit the local change.

```
git add README.md
git commit -m "Local conflicting change"
```

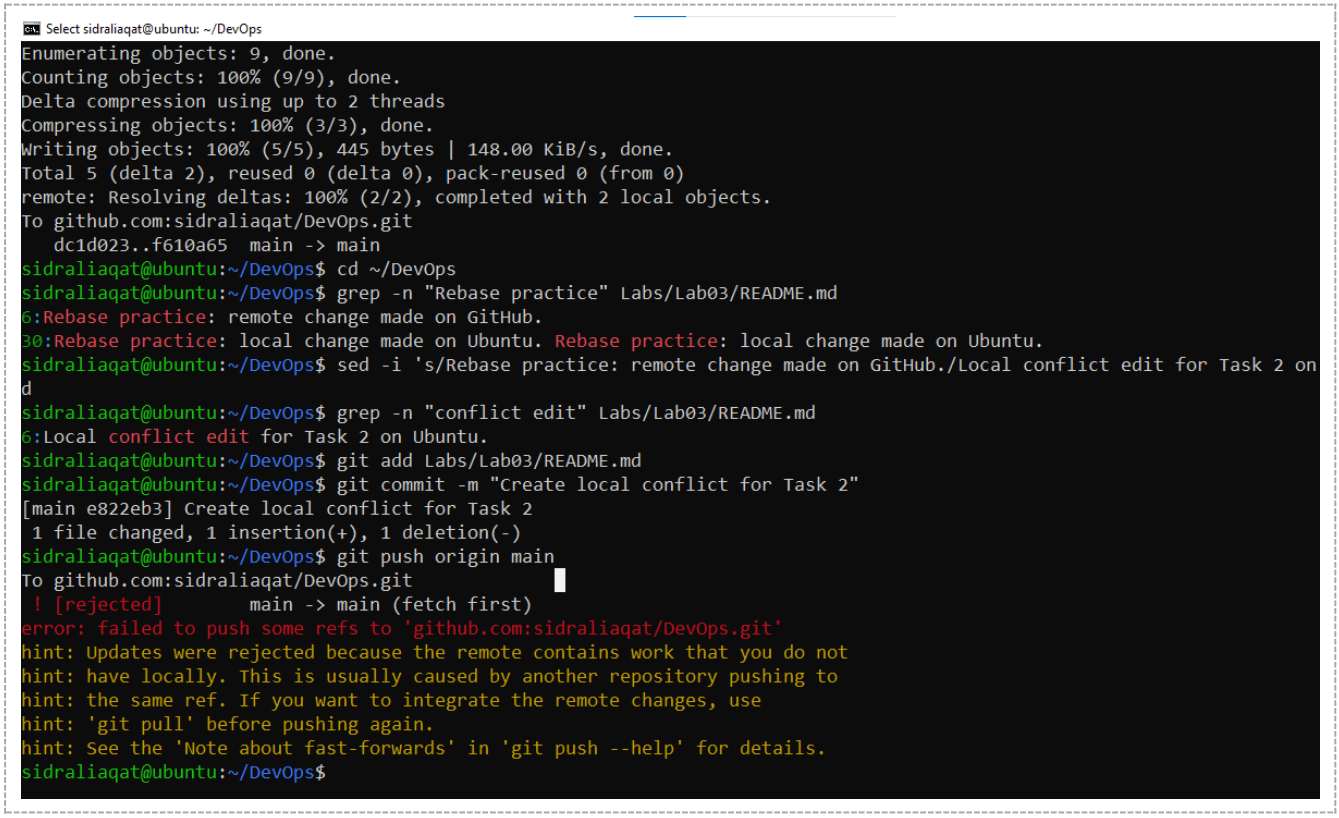
Screenshot: local_conflict_commit.png



4. Try to push. It gets rejected.

```
git push -u origin main
```

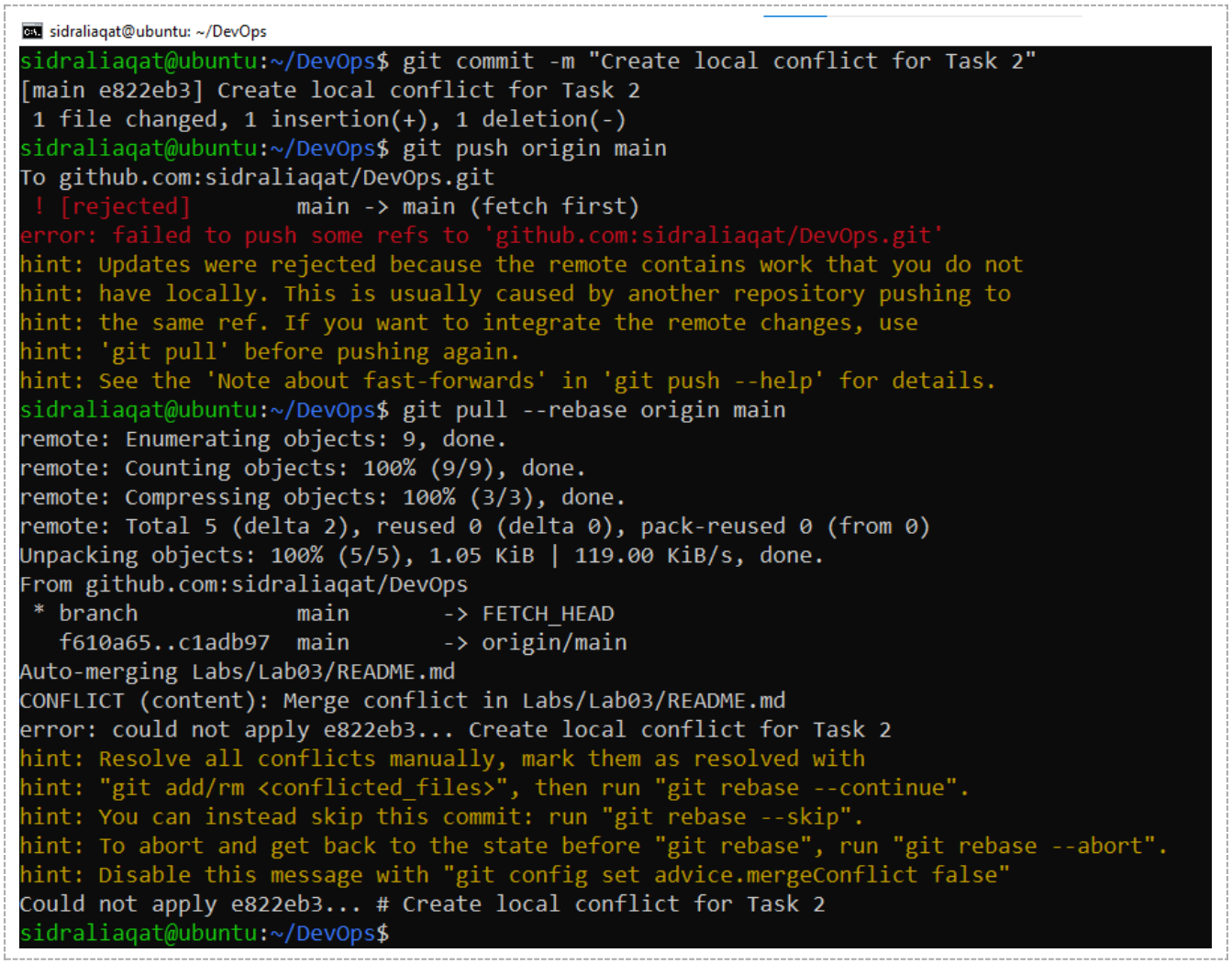
Screenshot: conflict_push_error.png



5. Pull with rebase. Git pauses and reports the conflict.

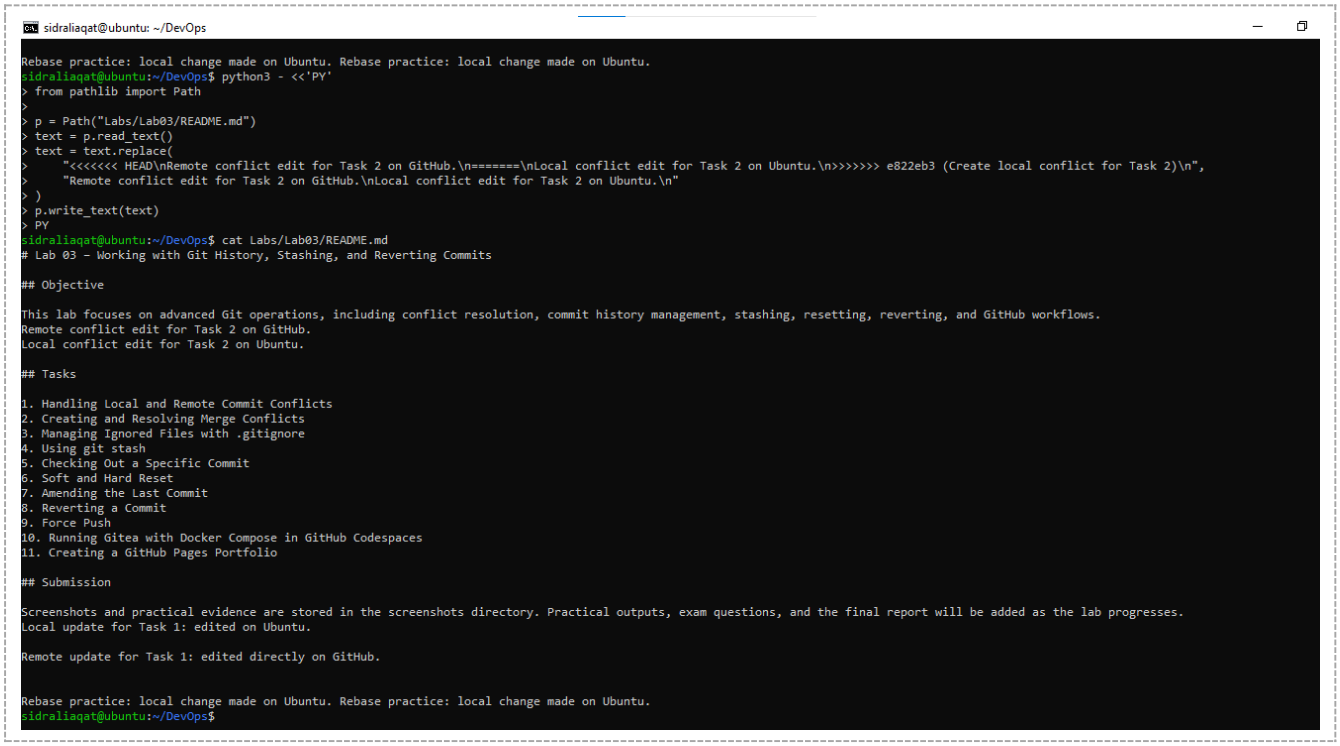
```
git pull --rebase origin main
```

Screenshot: conflict_message.png



6. Open README.md, remove the conflict markers, and keep the correct line (or merge both ideas).

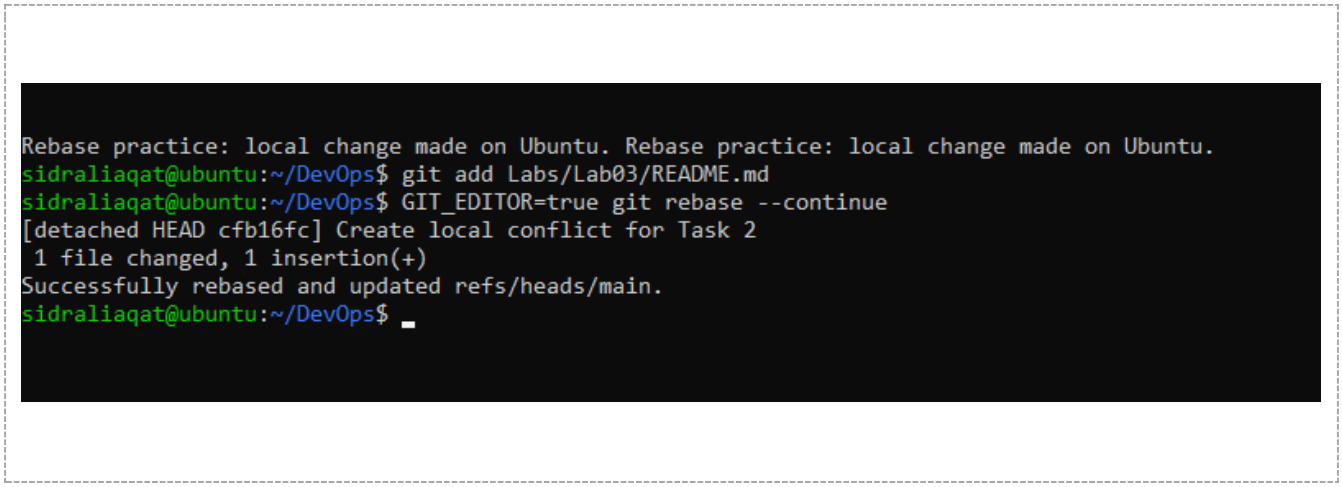
Screenshot: resolved_readme.png



7. Mark the conflict as resolved and continue the rebase.

```
git add README.md
git rebase --continue
```

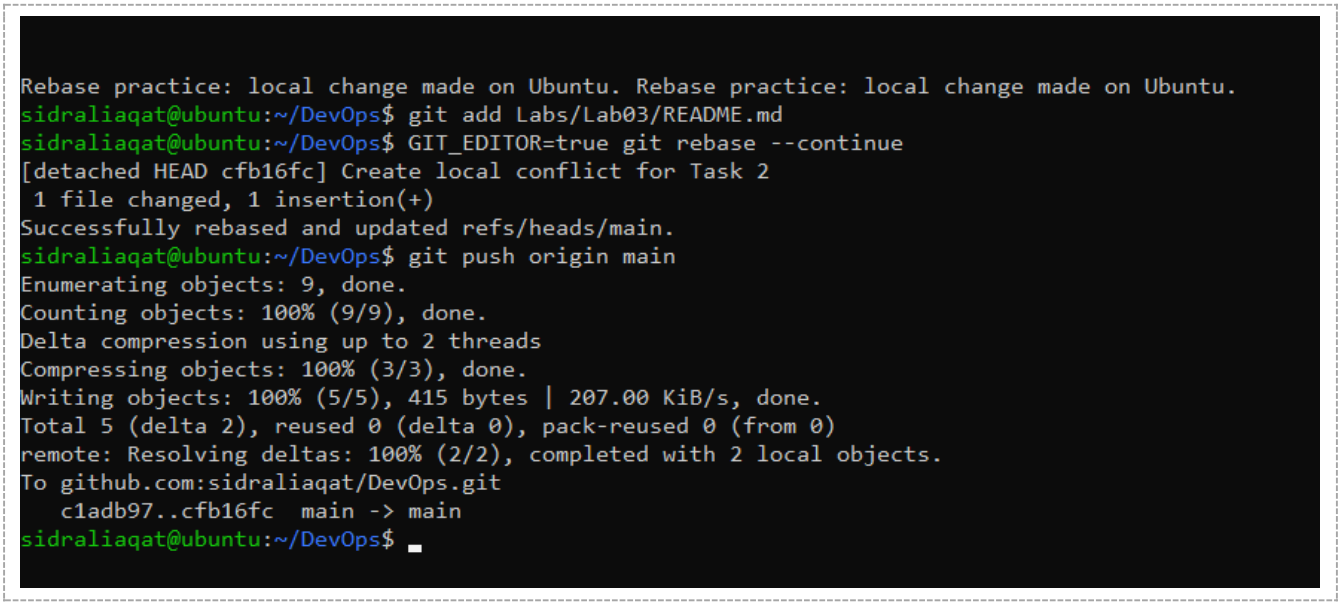
Screenshot: rebase_continue.png



8. Push the result.

```
git push -u origin main
```

Screenshot: **push_after_resolve.png**



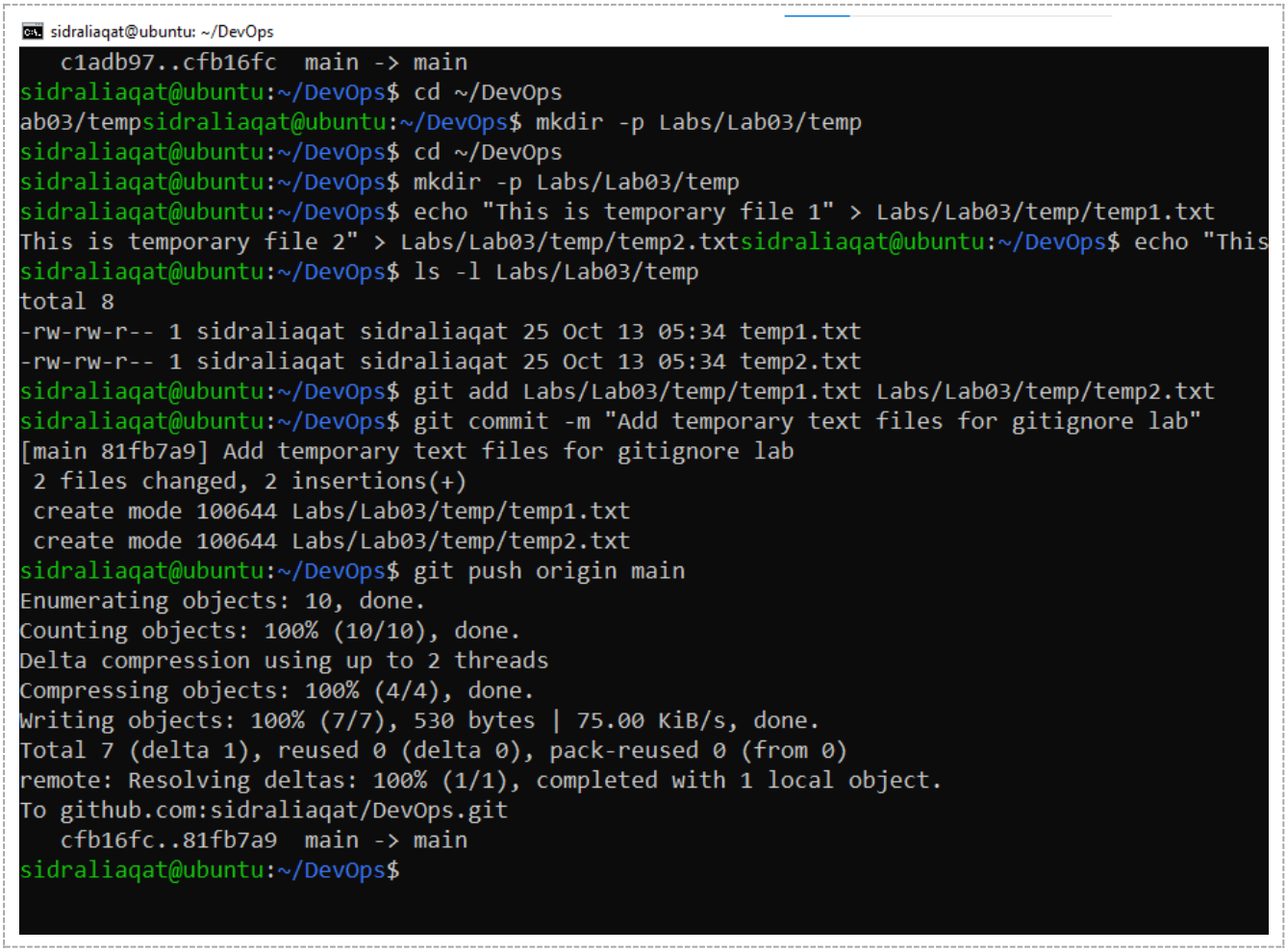
Task 3: Managing Ignored Files with .gitignore and Removing Tracked Files

A .gitignore file does not stop Git from tracking files it already tracks. git rm --cached removes them from the repo but keeps the local copies.

1. Create the textfiles folder with three files, then commit and push.

```
mkdir textfiles
echo "File A content" > textfiles/a.txt
echo "File B content" > textfiles/b.txt
echo "File C content" > textfiles/c.txt
git add .
git commit -m "Added textfiles directory with three files"
git push origin main
```

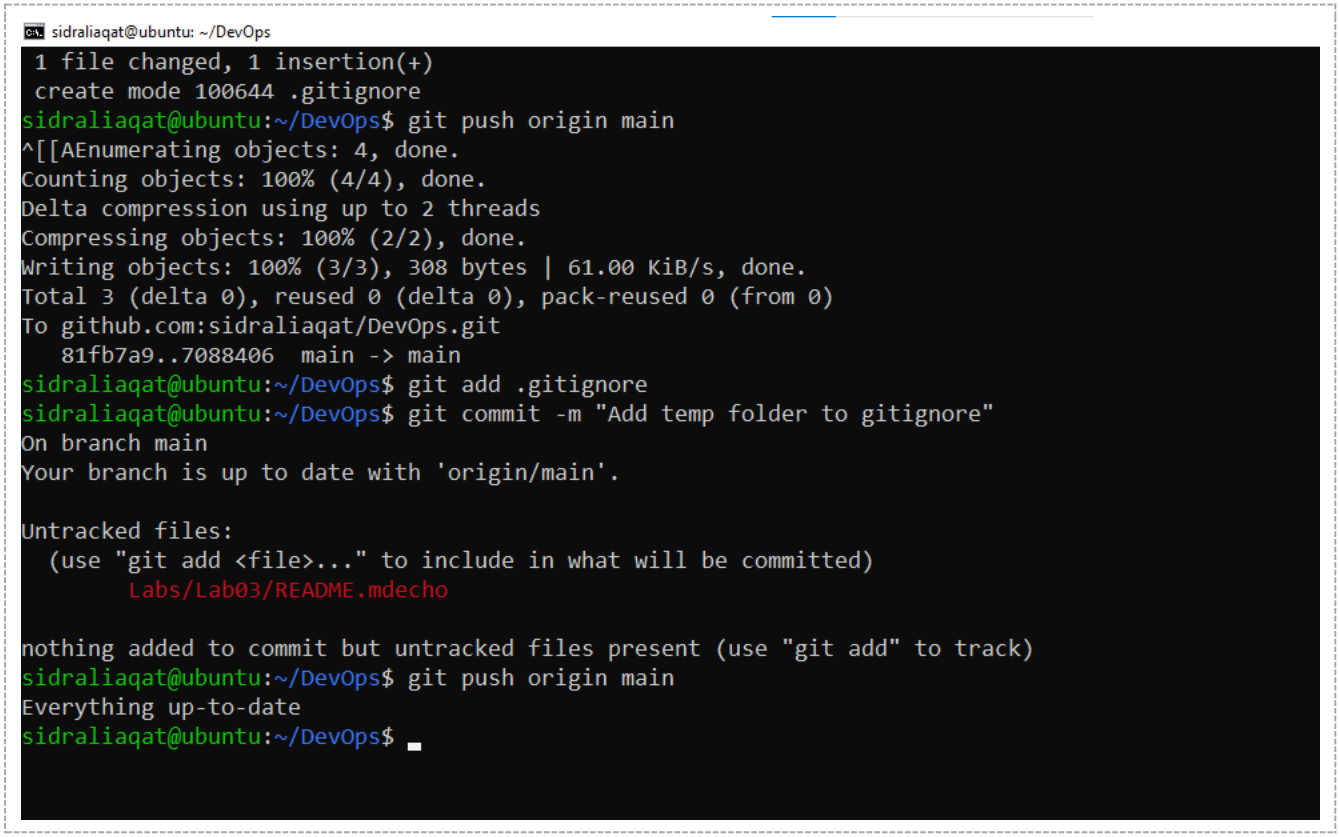
Screenshot: **push_textfiles.png**



2. Create .gitignore, then commit and push it.

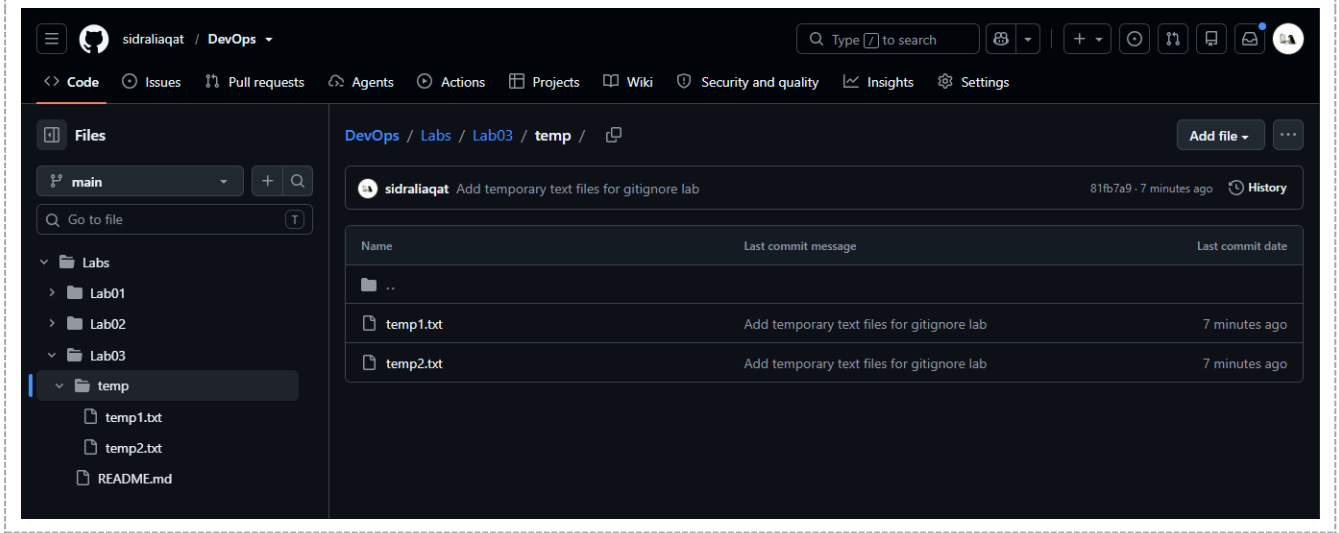
```
echo "textfiles/*" > .gitignore
git add .gitignore
git commit -m "Added .gitignore to ignore textfiles directory"
git push origin main
```

Screenshot: gitignore_push.png



3. Check GitHub. The textfiles folder is still there.

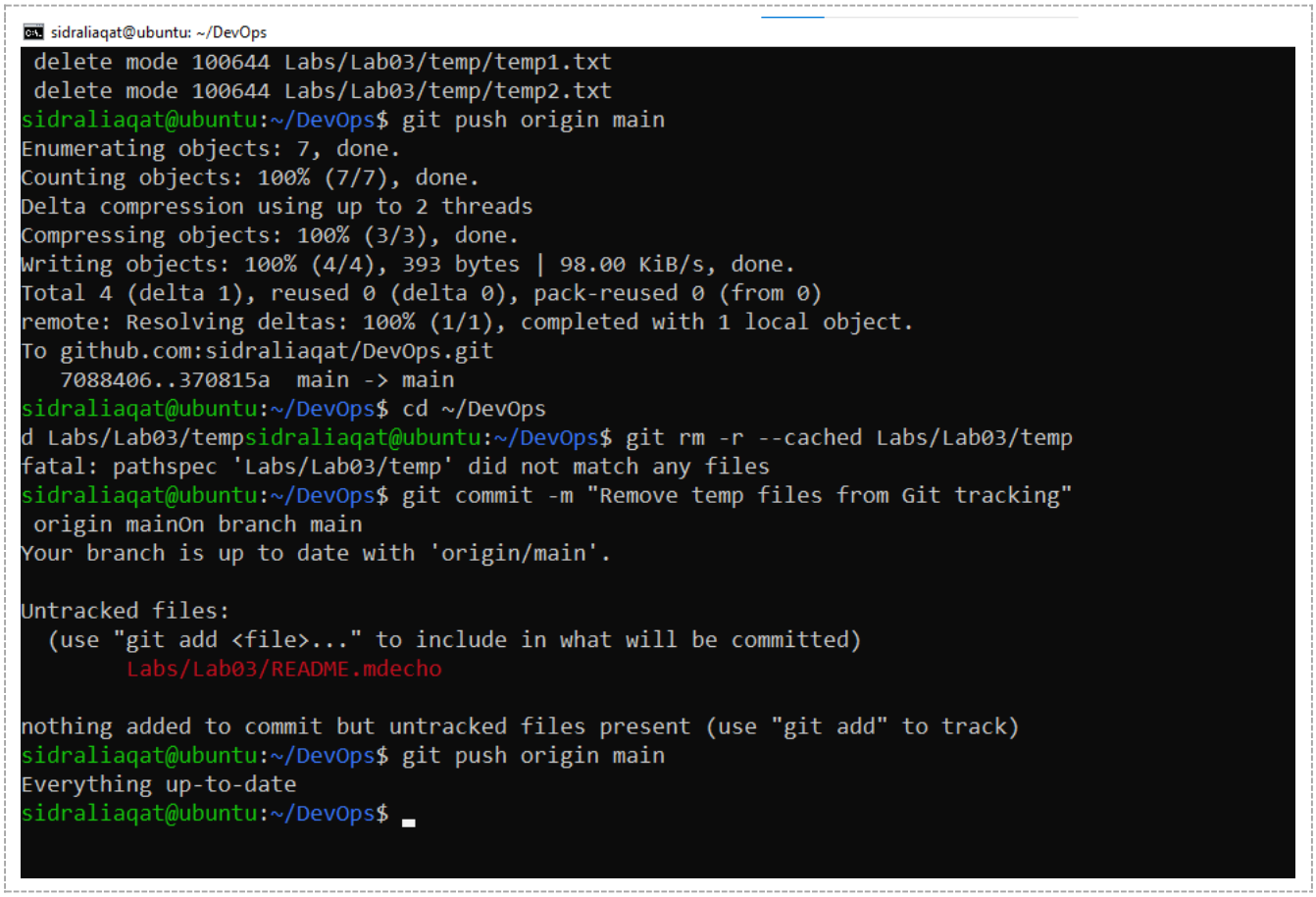
Screenshot: repo_still_has_textfiles.png



4. Remove the folder from tracking, then commit and push.

```
git rm -r --cached textfiles
git add .
git commit -m "Removed tracked textfiles directory"
git push origin main
```

Screenshot: rm_cached_push.png



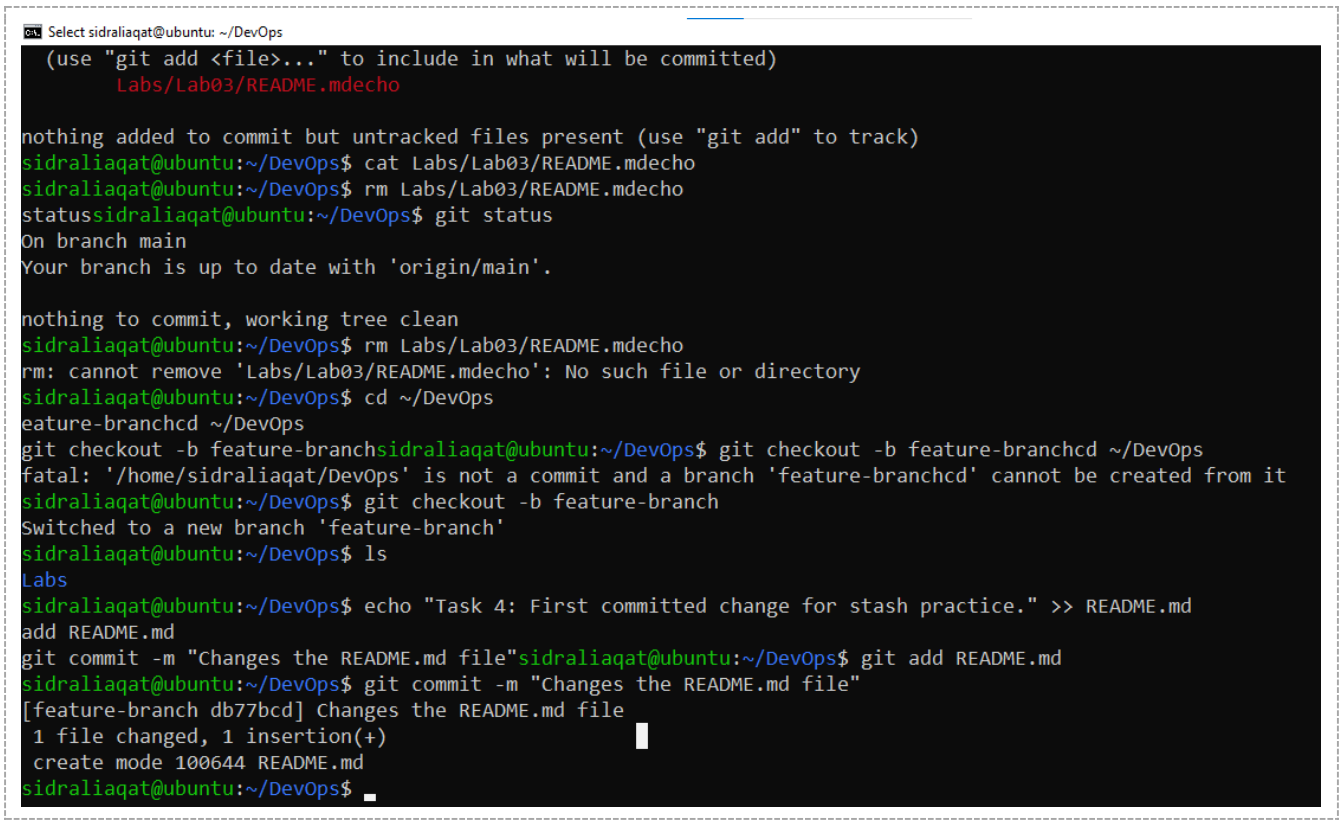
Task 4: Create Temporary Changes and Use git stash

git stash saves uncommitted work so you can switch branches without losing it.

1. Create feature-branch, edit README.md, and commit the change.

```
git checkout -b feature-branch
git add README.md
git commit -m "Changes the README.md file"
```

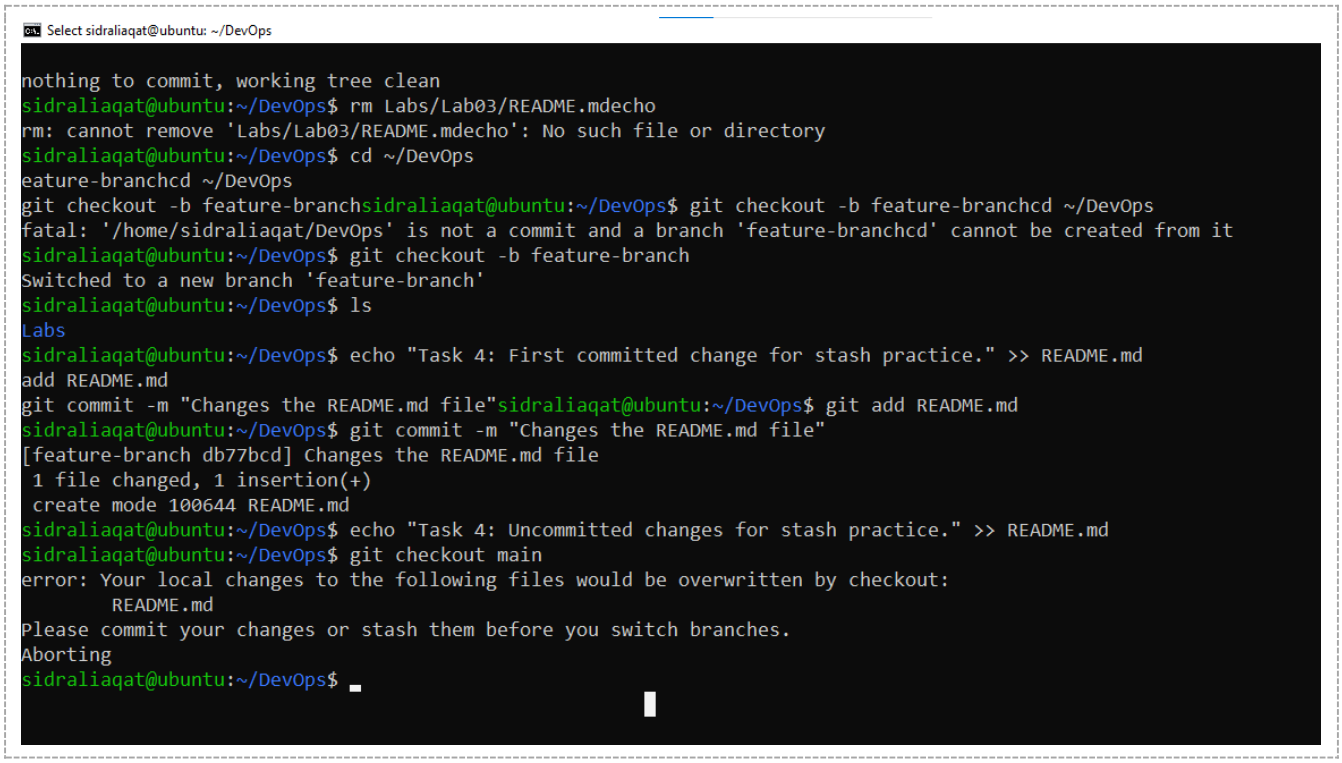
Screenshot: modified_readme.png



2. Edit README.md again without committing, then try to switch to main. Git blocks the checkout.

```
git checkout main
```

Screenshot: checkout_error.png



3. Stash the changes.

```
git stash
```

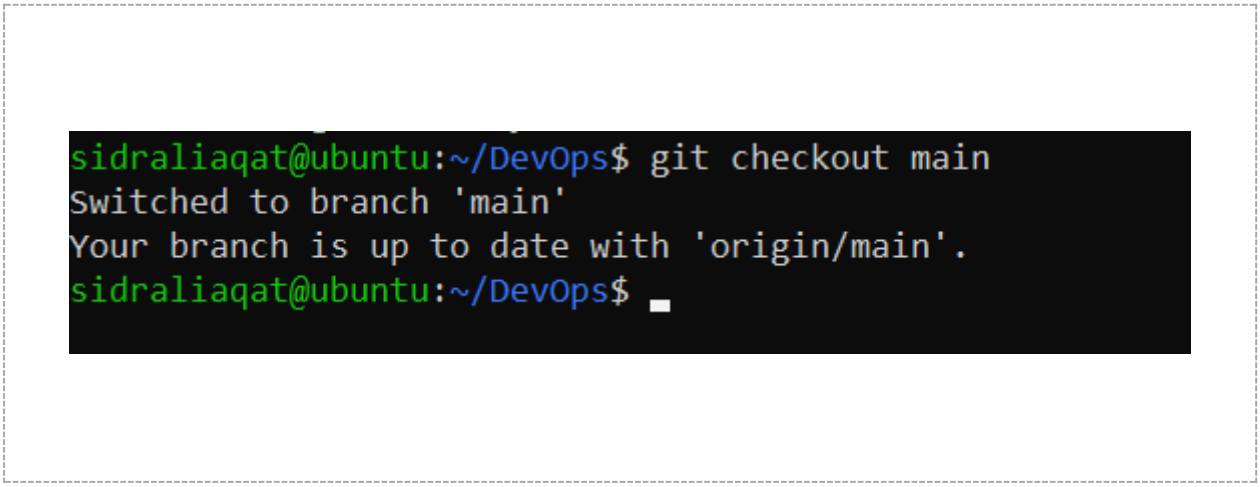
Screenshot: stash_command.png



4. Switch to main again.

```
git checkout main
```

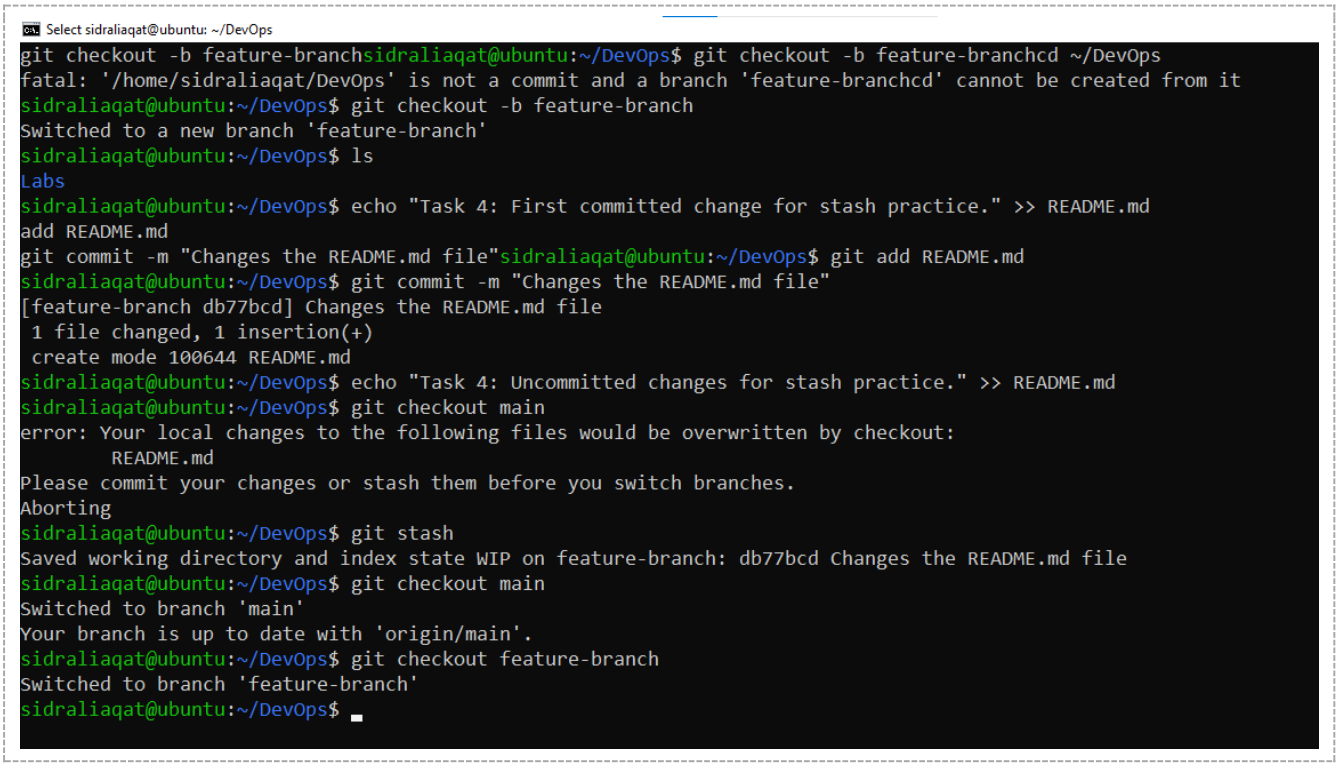
Screenshot: branch_switched.png



5. Go back to feature-branch.

```
git checkout feature-branch
```

Screenshot: back_to_feature.png



6. Check the working directory.

```
git status
```

Screenshot: status_clean.png

```
sidraliaqat@ubuntu:~/DevOps$ git stash
Saved working directory and index state WIP on feature-branch: db77bcd Changes the README.md file
sidraliaqat@ubuntu:~/DevOps$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
sidraliaqat@ubuntu:~/DevOps$ git checkout feature-branch
Switched to branch 'feature-branch'
sidraliaqat@ubuntu:~/DevOps$ git status
On branch feature-branch
nothing to commit, working tree clean
sidraliaqat@ubuntu:~/DevOps$
```

7. Restore the stashed changes and confirm the edits are back.

```
git stash pop
```

Screenshot: stash_pop.png

```
sidraliaqat@ubuntu: ~/DevOps
[feature-branch db77bcd] Changes the README.md file
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
sidraliaqat@ubuntu:~/DevOps$ echo "Task 4: Uncommitted changes for stash practice." >> README.md
sidraliaqat@ubuntu:~/DevOps$ git checkout main
error: Your local changes to the following files would be overwritten by checkout:
        README.md
Please commit your changes or stash them before you switch branches.
Aborting
sidraliaqat@ubuntu:~/DevOps$ git stash
Saved working directory and index state WIP on feature-branch: db77bcd Changes the README.md file
sidraliaqat@ubuntu:~/DevOps$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
sidraliaqat@ubuntu:~/DevOps$ git checkout feature-branch
Switched to branch 'feature-branch'
sidraliaqat@ubuntu:~/DevOps$ git status
On branch feature-branch
nothing to commit, working tree clean
sidraliaqat@ubuntu:~/DevOps$ git stash pop
On branch feature-branch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (bc8bc78c11a66b852f5b82547167dd375f5ea0bd)
sidraliaqat@ubuntu:~/DevOps$
```

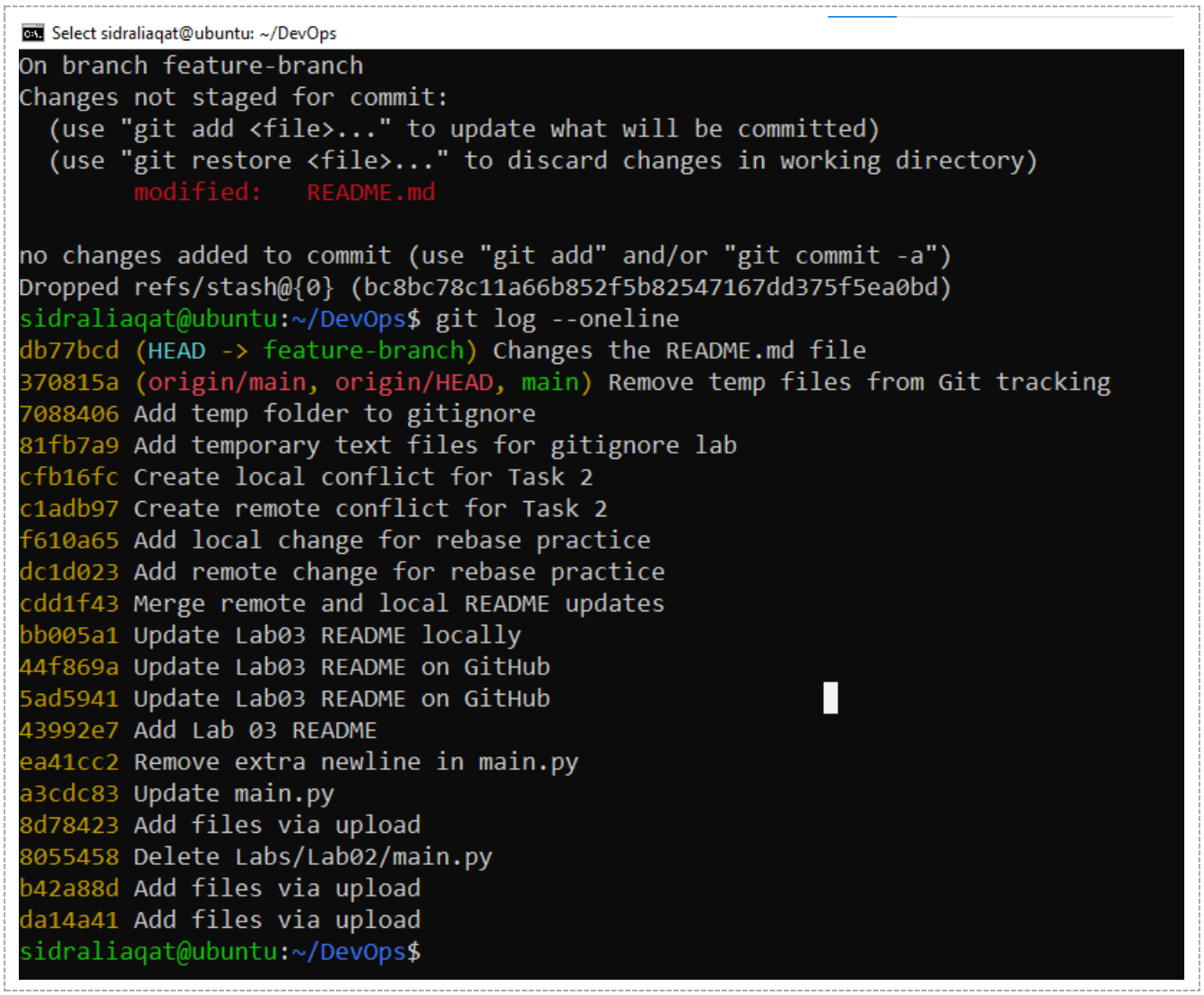
Task 5: Checkout a Specific Commit Using git log

Checking out a commit hash puts you in detached HEAD state, where changes are not attached to any branch.

1. View the commit history.

```
git log --oneline
```

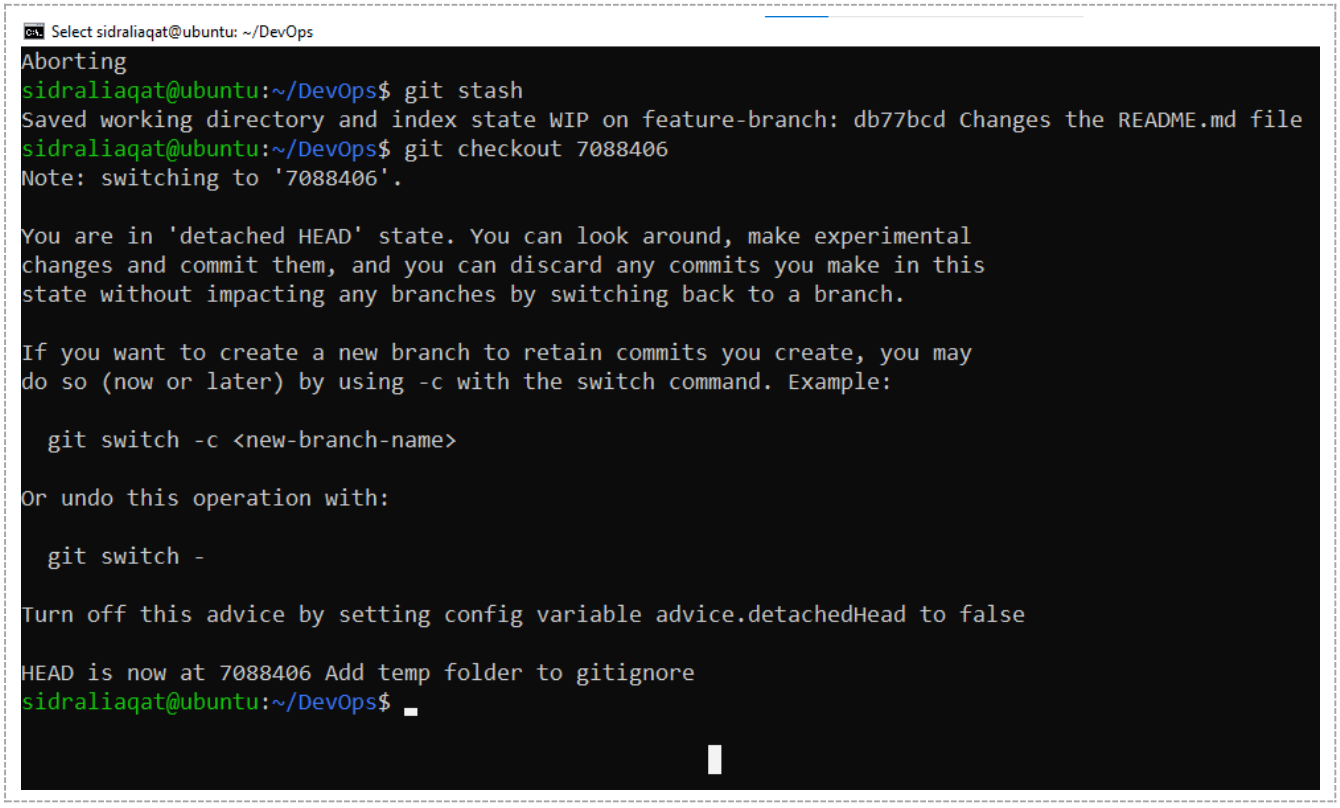
Screenshot: log_before_checkout.png



2. Copy an earlier commit hash and check it out.

```
git checkout <commit-hash>
```

Screenshot: detached_head.png



3. Return to main.

```
git checkout main
```

Screenshot: back_to_main.png



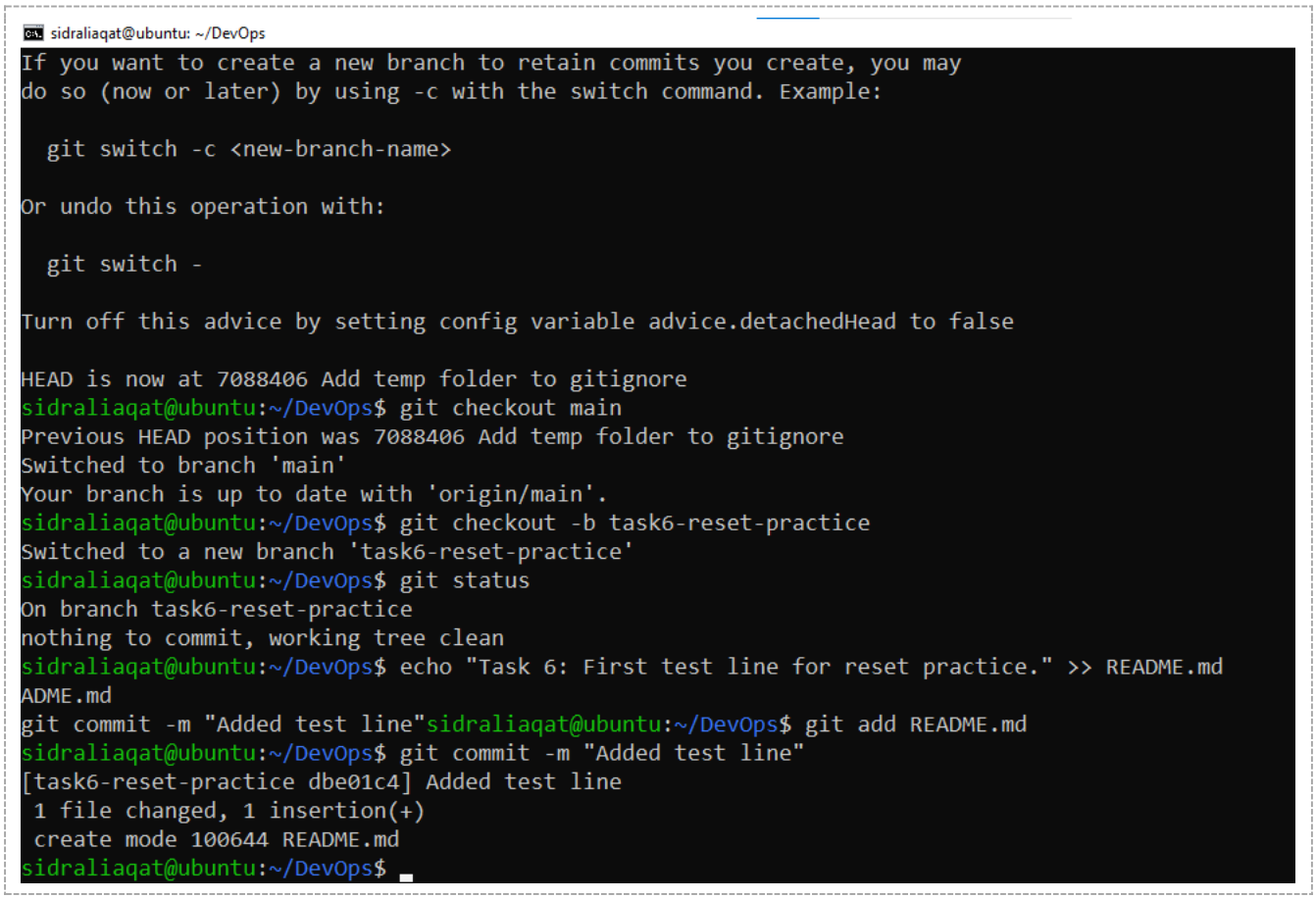
Task 6: Resetting Commits (Soft vs Hard Reset)

A soft reset keeps your changes staged. A hard reset throws them away. History and file contents are checked after each reset.

1. Add a line to a file and commit it.

```
git add .
git commit -m "Added test line"
```

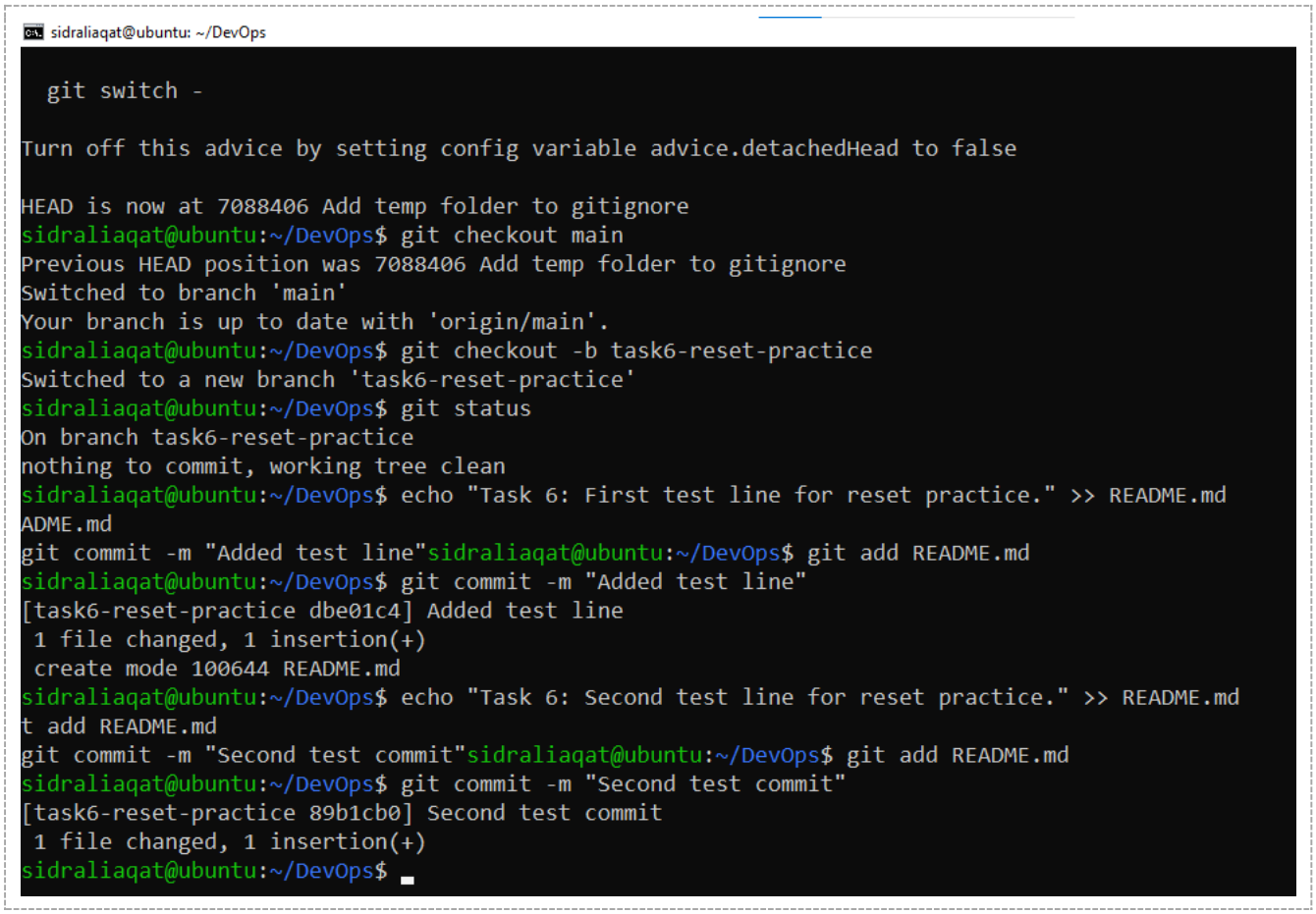
Screenshot: first_commit.png



2. Add another line and commit again.

```
git add .
git commit -m "Second test commit"
```

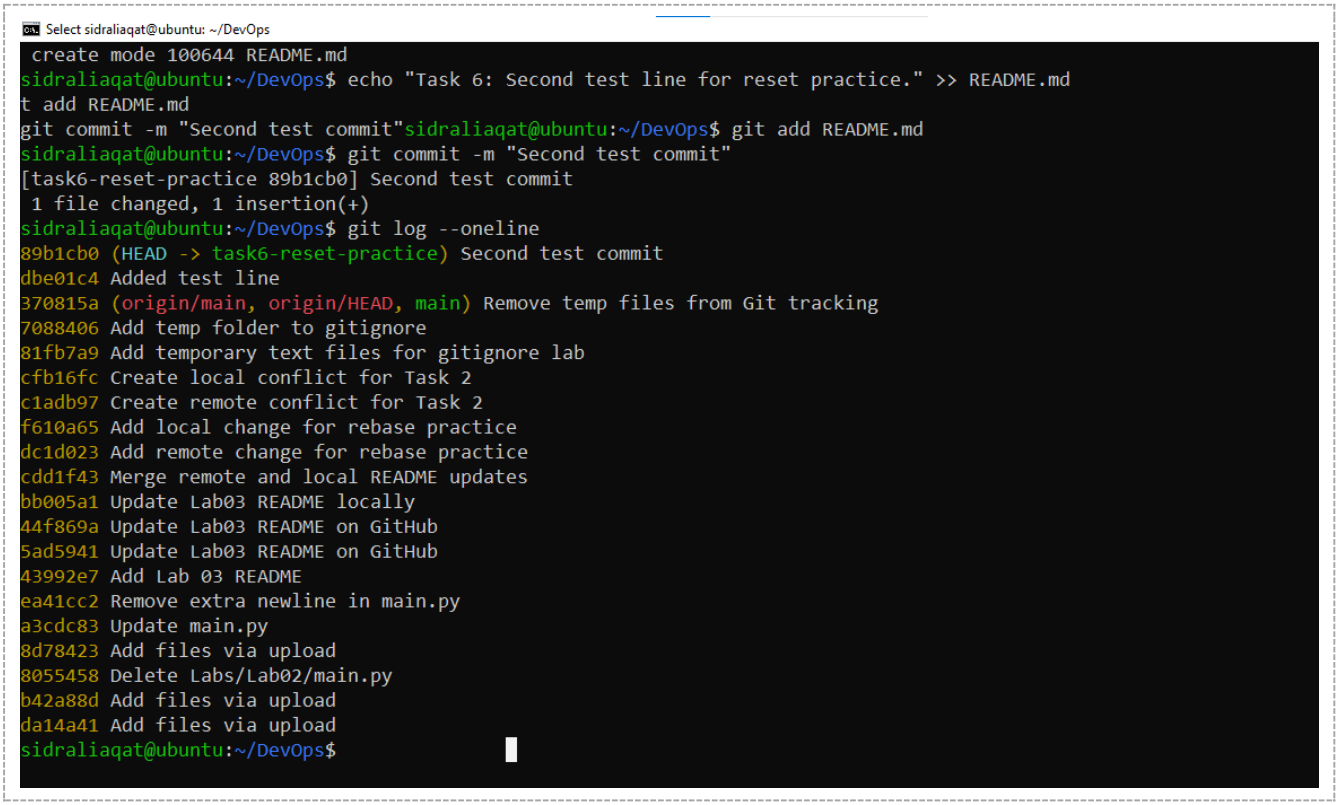
Screenshot: second_commit.png



3. View the history before the reset.

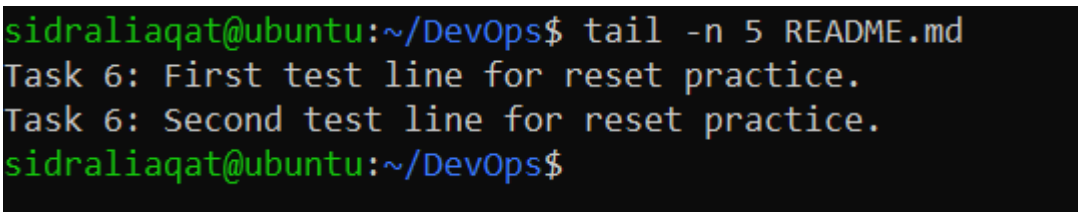
```
git log --oneline
```

Screenshot: log_before_reset.png



4. Open the edited file and confirm both added lines are there.

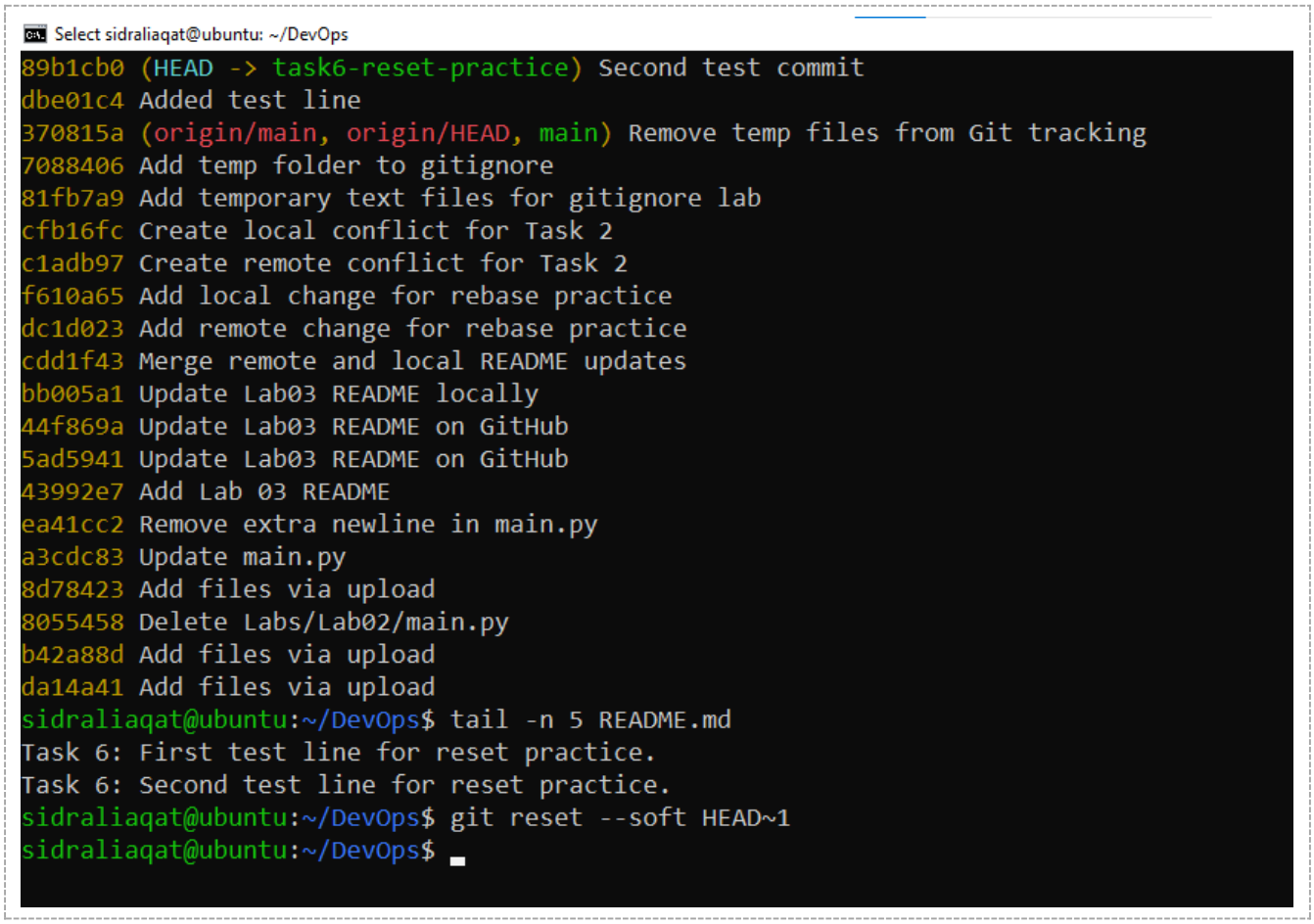
Screenshot: file_before_reset.png



5. Do a soft reset.

```
git reset --soft HEAD~1
```

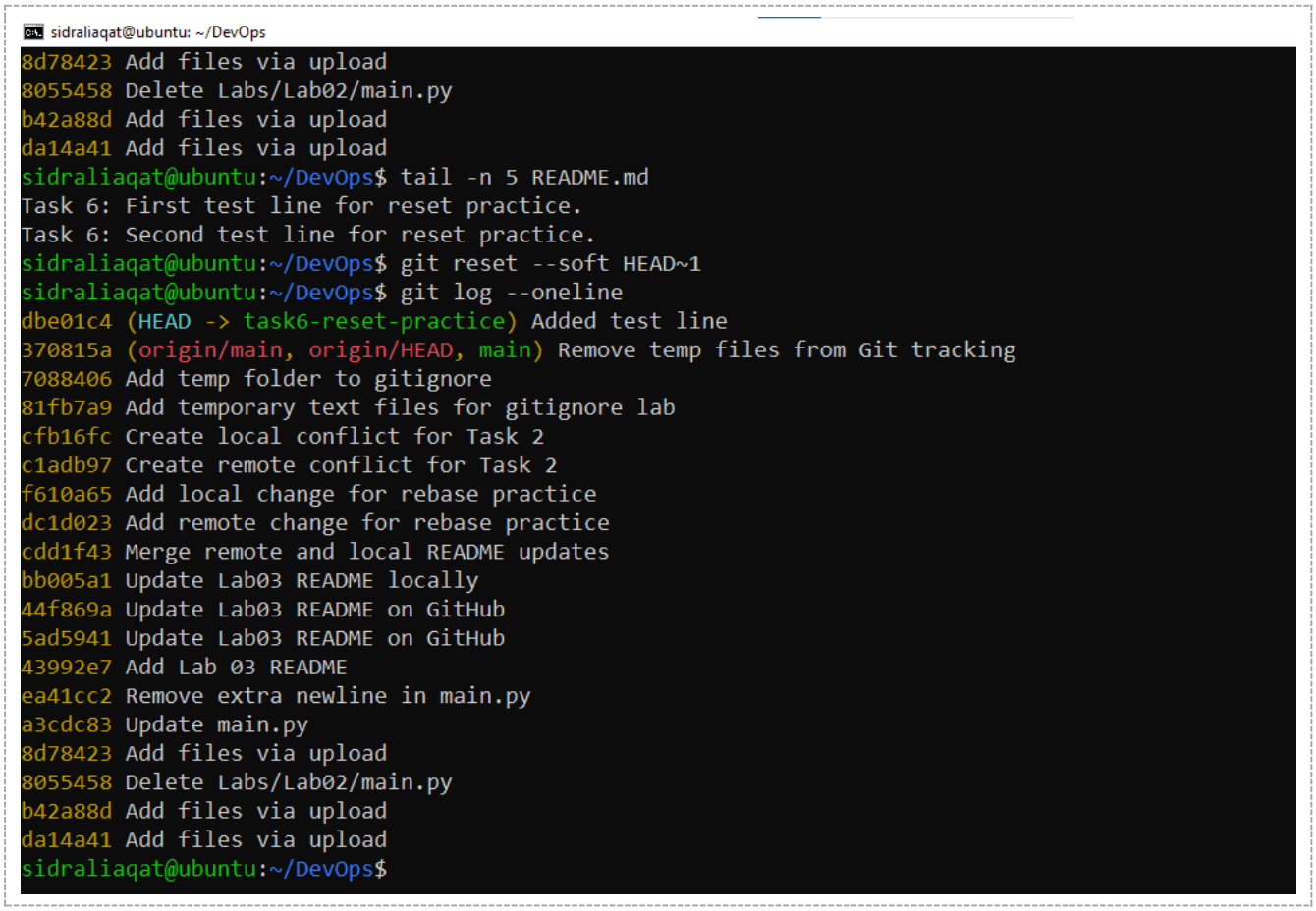
Screenshot: soft_reset.png



6. Check the history after the soft reset.

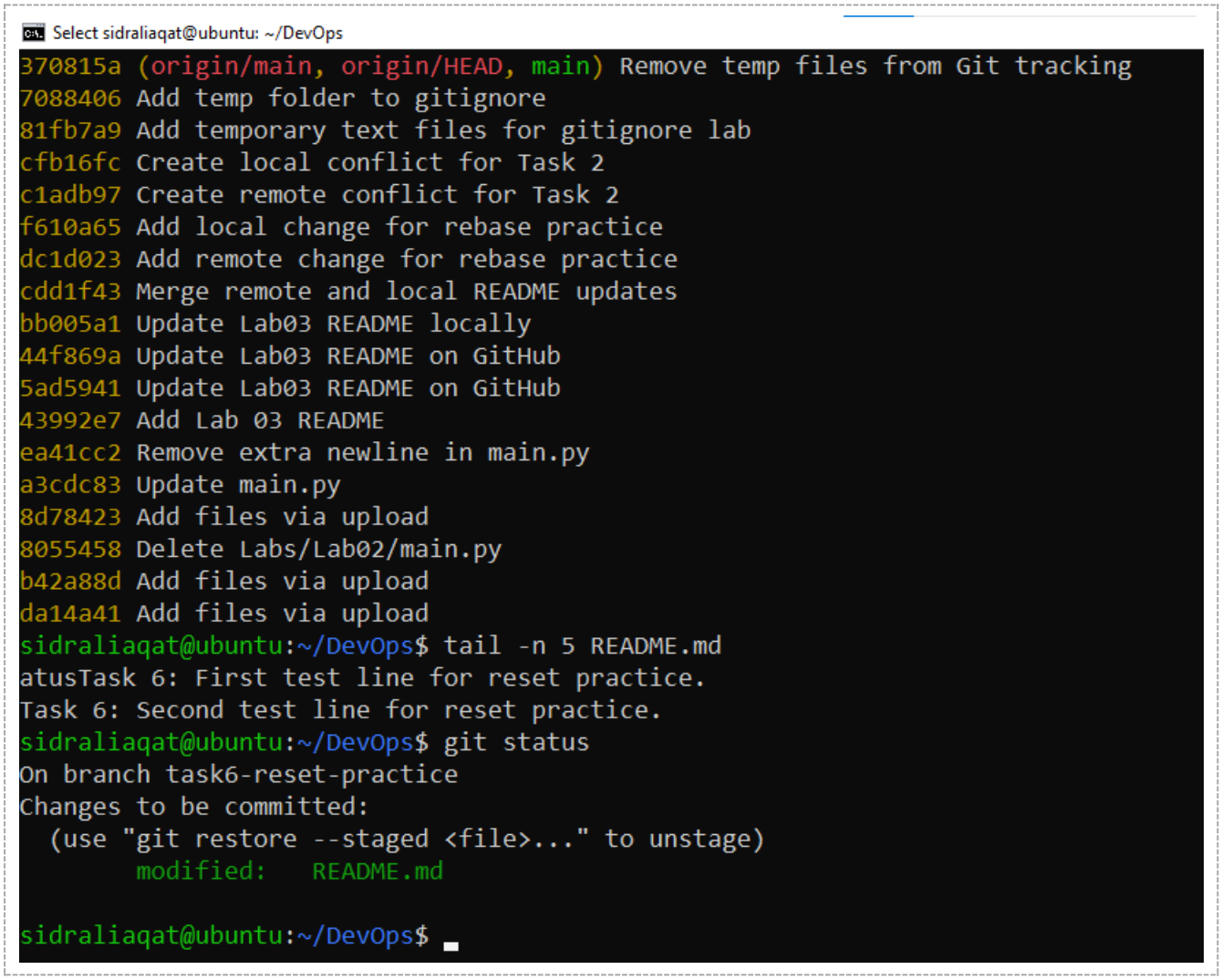
```
git log --oneline
```

Screenshot: log_after_soft_reset.png



7. Open the file again. Both edits should still be there.

Screenshot: file_after_soft_reset.png



8. Check git status. The changes should be staged.

```
git status
```

9. Commit again.

```
git commit -m "Second Test commit"
```

Screenshot: recommit_after_soft_reset.png

10. Do a hard reset.

```
git reset --hard HEAD~1
```

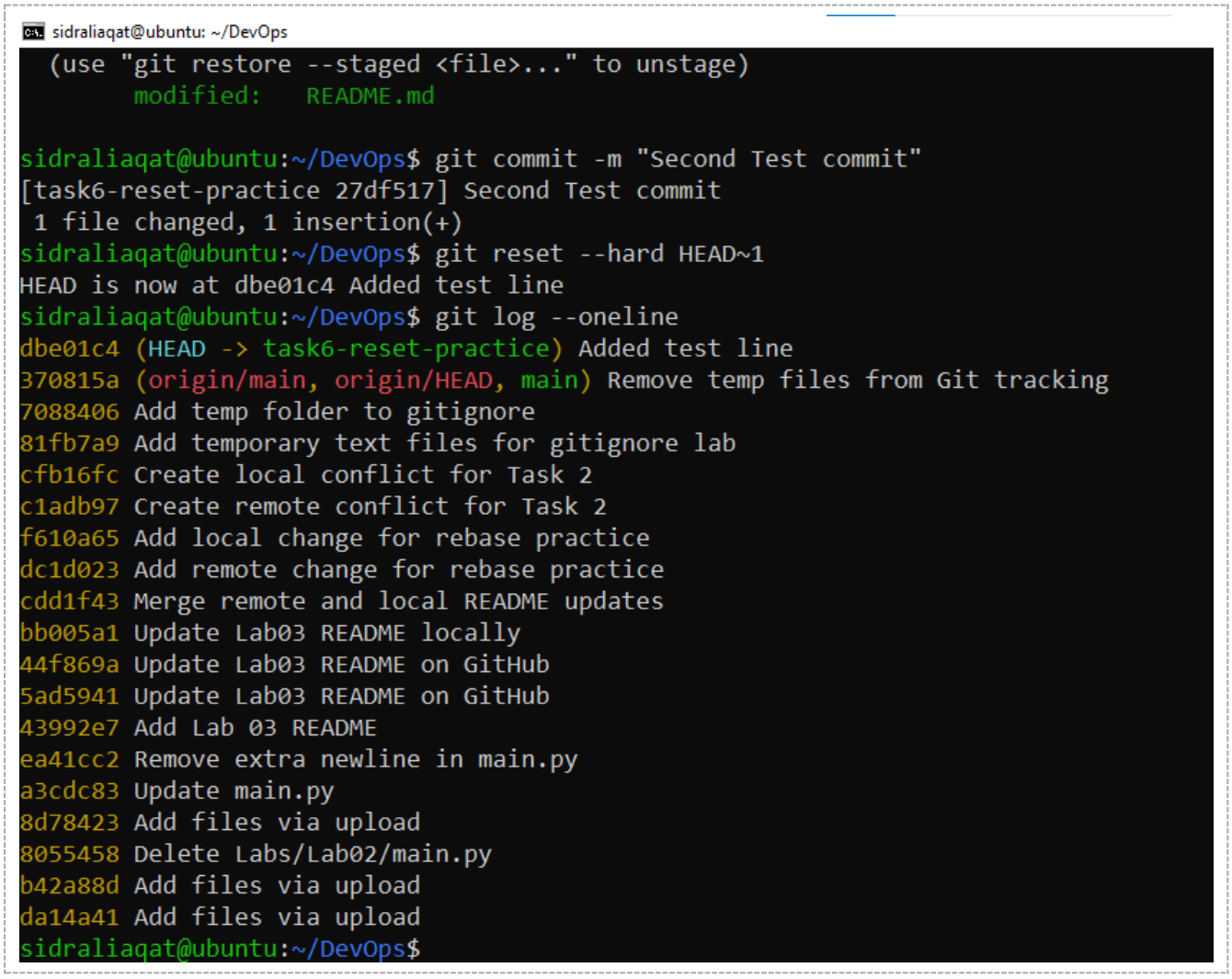
Screenshot: hard_reset.png



11. Check the history after the hard reset.

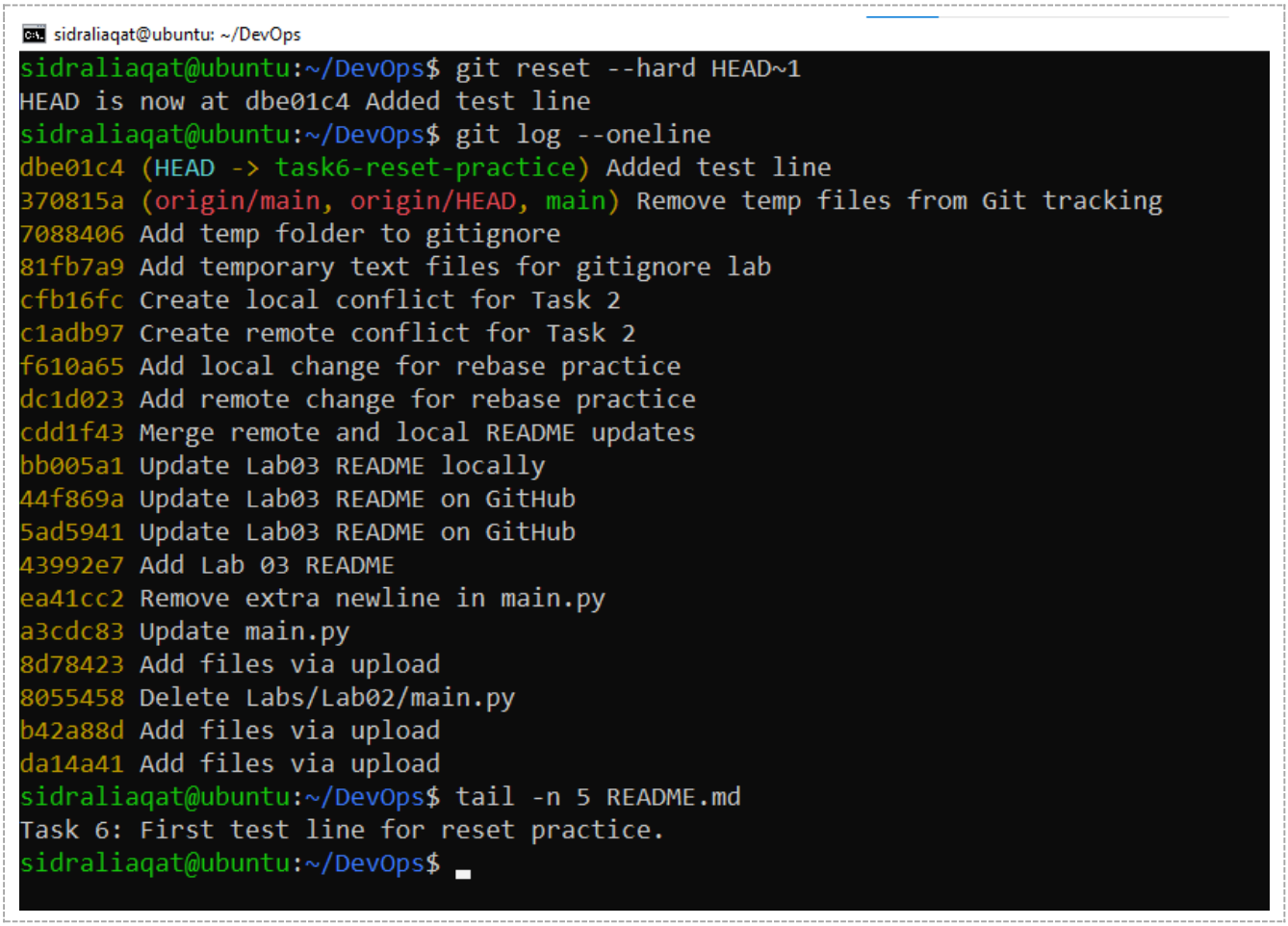
```
git log --oneline
```

Screenshot: log_after_hard_reset.png



12. Open the file. The second edit should be gone.

Screenshot: file_after_hard_reset.png



13. Check git status. It should say the working tree is clean.

```
git status
```

Task 7: Amending the Last Commit

git commit --amend changes the last commit, either its message or the files in it.

1. Make a small change, then stage and commit it.

```
git add .
git commit -m "Fix log message"
```

Screenshot: first_amend_commit.png

```
sidraliaqat@ubuntu: ~/DevOps
5ad5941 Update Lab03 README on GitHub
43992e7 Add Lab 03 README
ea41cc2 Remove extra newline in main.py
a3cdc83 Update main.py
8d78423 Add files via upload
8055458 Delete Labs/Lab02/main.py
b42a88d Add files via upload
da14a41 Add files via upload
sidraliaqat@ubuntu:~/DevOps$ tail -n 5 README.md
Task 6: First test line for reset practice.
sidraliaqat@ubuntu:~/DevOps$ git status
On branch task6-reset-practice
nothing to commit, working tree clean
sidraliaqat@ubuntu:~/DevOps$ git log --oneline -3
dbe01c4 (HEAD -> task6-reset-practice) Added test line
370815a (origin/main, origin/HEAD, main) Remove temp files from Git tracking
7088406 Add temp folder to gitignore
sidraliaqat@ubuntu:~/DevOps$ echo "Task 7: Amended commit practice." >> README.md
sidraliaqat@ubuntu:~/DevOps$ git add README.md
it -m "First amend commit"sidraliaqat@ubuntu:~/DevOps$ git commit -m "First amend commit"
[task6-reset-practice ee81fe3] First amend commit
 1 file changed, 1 insertion(+)
sidraliaqat@ubuntu:~/DevOps$
```

2. Update the file you forgot, stage it, and amend the last commit.

```
git add .
git commit --amend
```

Screenshot: amend_commit.png

```
sidraliaqat@ubuntu:~/DevOps$ git log --oneline -3
dbe01c4 (HEAD -> task6-reset-practice) Added test line
370815a (origin/main, origin/HEAD, main) Remove temp files from Git tracking
7088406 Add temp folder to gitignore
sidraliaqat@ubuntu:~/DevOps$ echo "Task 7: Amended commit practice." >> README.md
sidraliaqat@ubuntu:~/DevOps$ git add README.md
it -m "First amend commit"sidraliaqat@ubuntu:~/DevOps$ git commit -m "First amend commit"
[task6-reset-practice ee81fe3] First amend commit
 1 file changed, 1 insertion(+)
sidraliaqat@ubuntu:~/DevOps$ echo "Task 7: Final amended line." >> README.md
sidraliaqat@ubuntu:~/DevOps$ git add README.md
git commit --amend -m "Amended commit"sidraliaqat@ubuntu:~/DevOps$ git commit --amend -m "Amended commit"
[task6-reset-practice 4013f59] Amended commit
 Date: Tue Oct 13 06:26:02 2026 +0000
 1 file changed, 2 insertions(+)
sidraliaqat@ubuntu:~/DevOps$
```

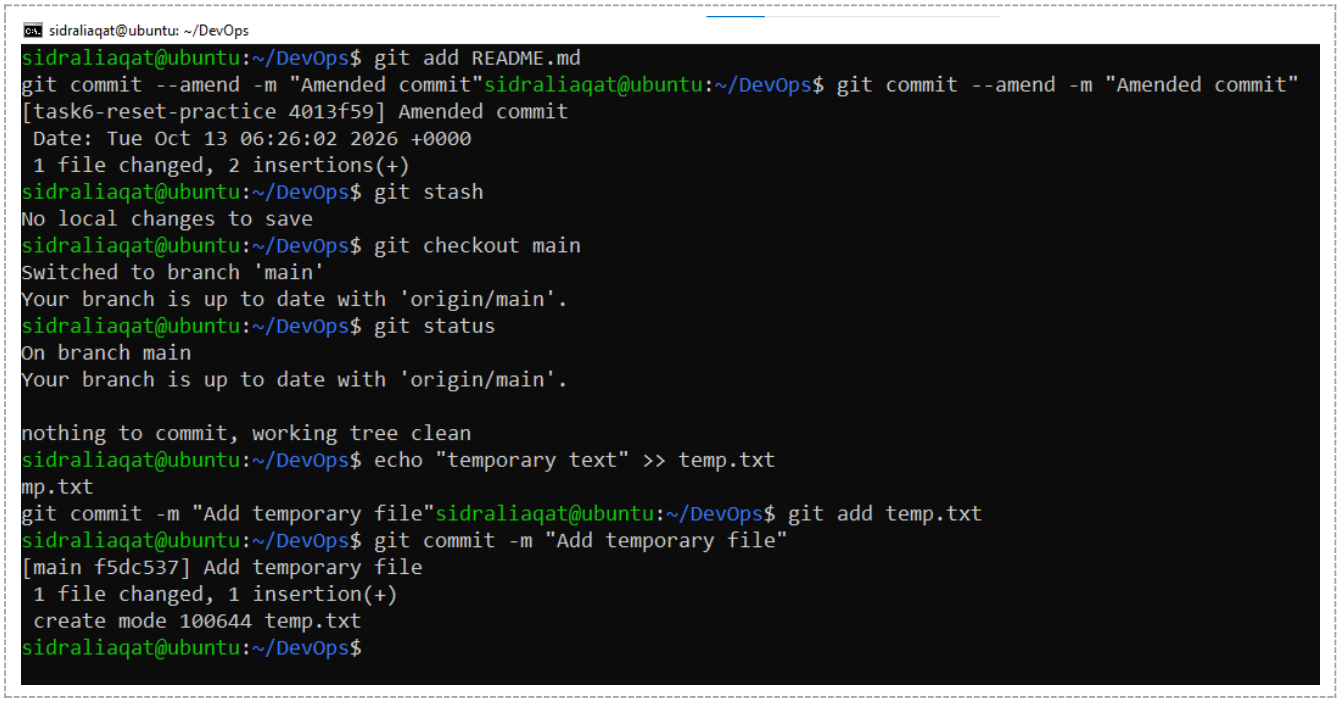
Task 8: Reverting a Commit (Safe Undo on Remote Branch)

git revert adds a new commit that undoes an earlier one, so shared history is never rewritten.

1. Add temp.txt content, then commit and push.

```
echo "temporary text" >> temp.txt
git add .
git commit -m "Added temporary text file"
git push origin main
```

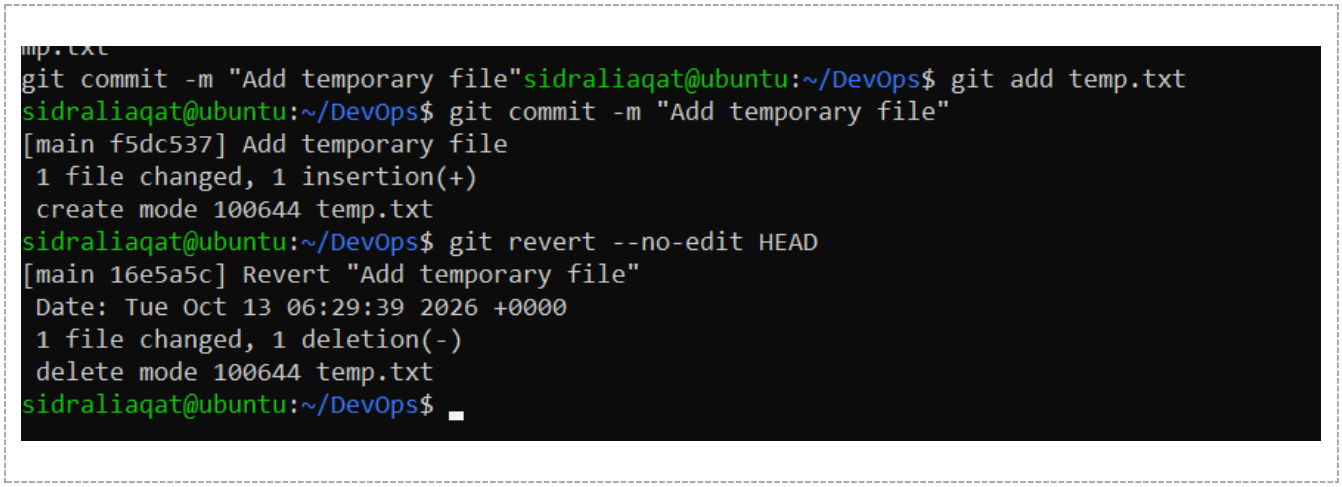
Screenshot: commit_temp_file.png



2. Find the commit hash and revert it.

```
git log --oneline
git revert <commit-hash>
```

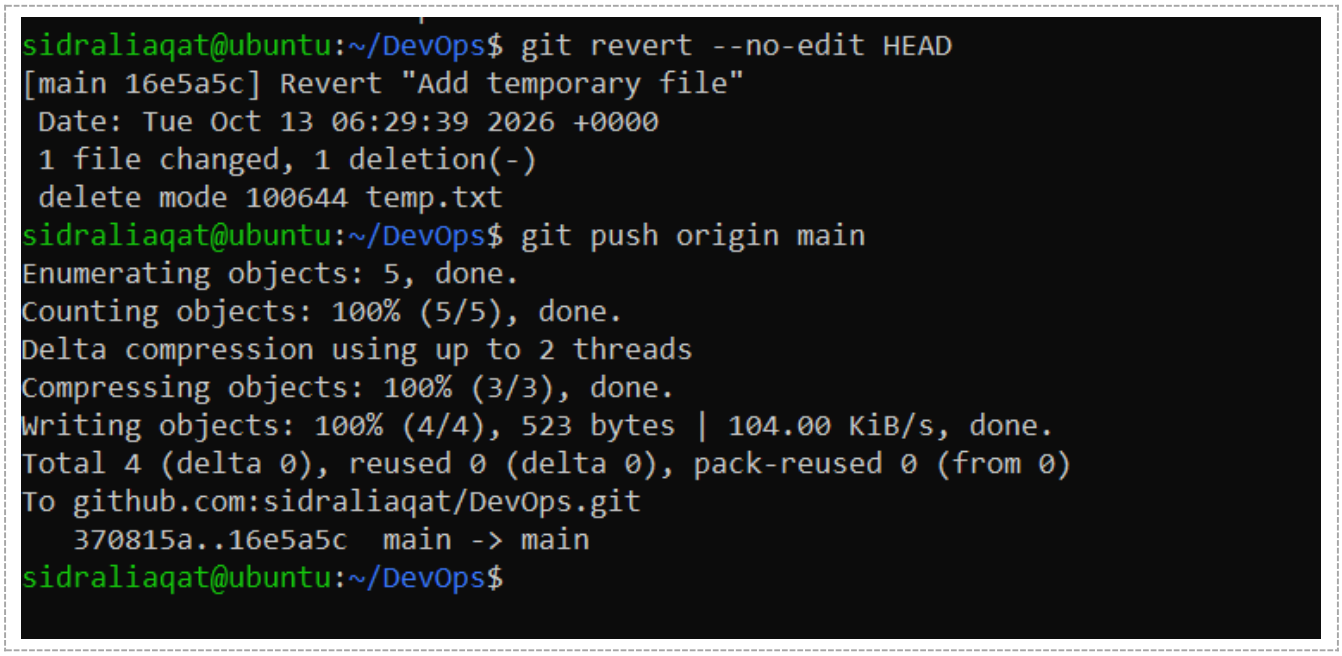
Screenshot: revert_commit.png



3. Push the revert commit.

```
git push origin main
```

Screenshot: revert_push.png



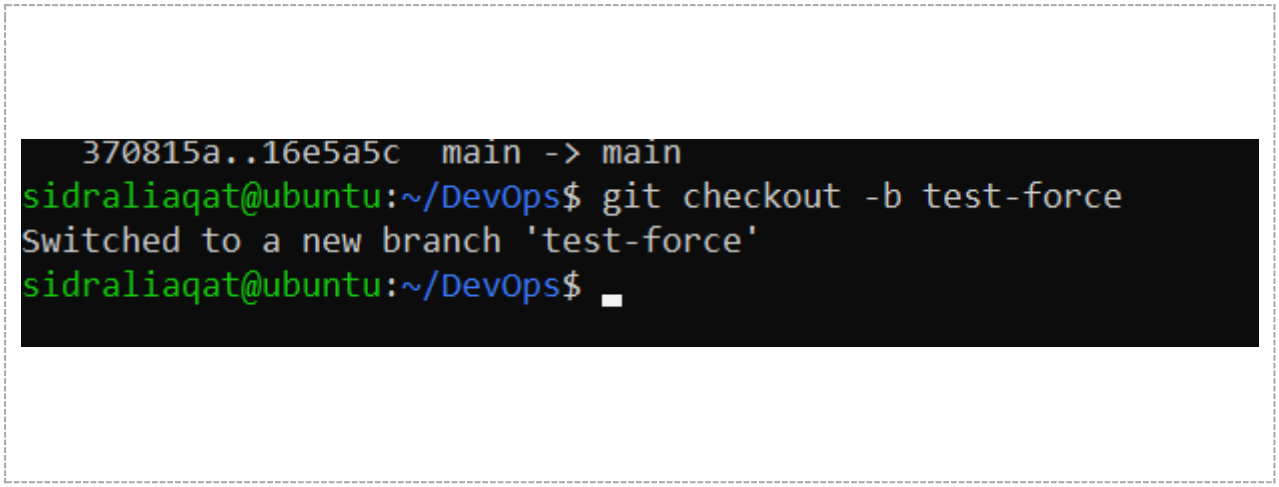
Task 9: Force Push (With Caution)

Force push rewrites remote history, so it is only used here on a personal test branch, never on main.

1. Create a new branch.

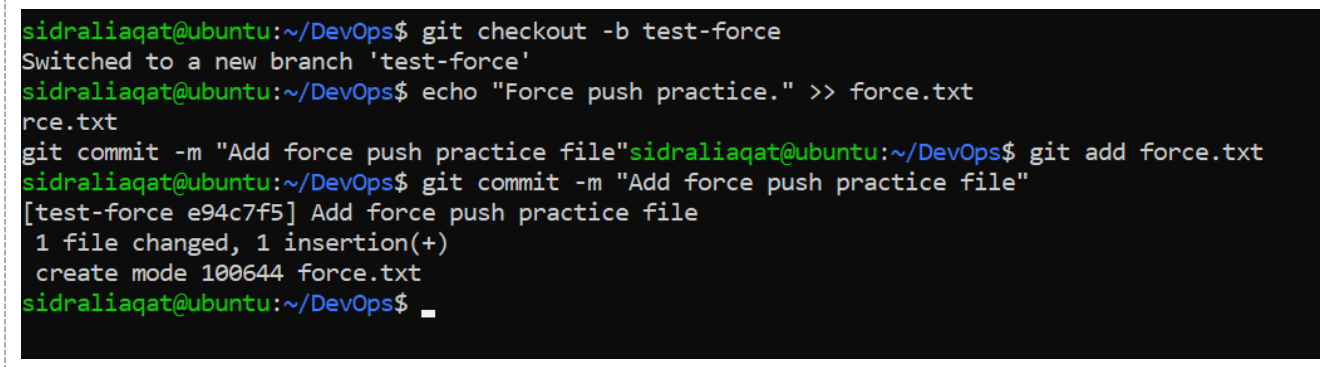
```
git checkout -b test-force
```

Screenshot: new_branch.png



2. Make and commit a small change.

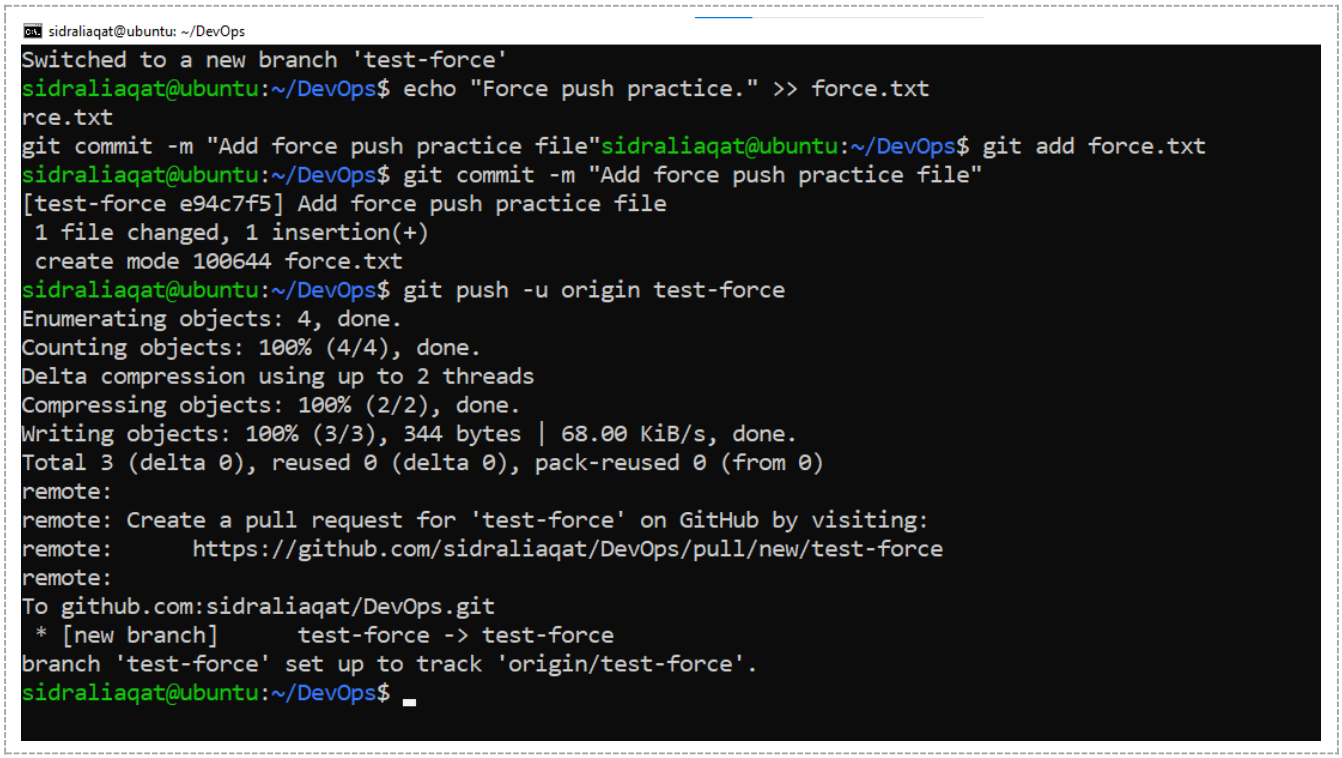
Screenshot: force_commit.png



3. Push the branch.

```
git push origin test-force
```

Screenshot: push_force_branch.png



4. Do a hard reset.

```
git reset --hard HEAD~1
```

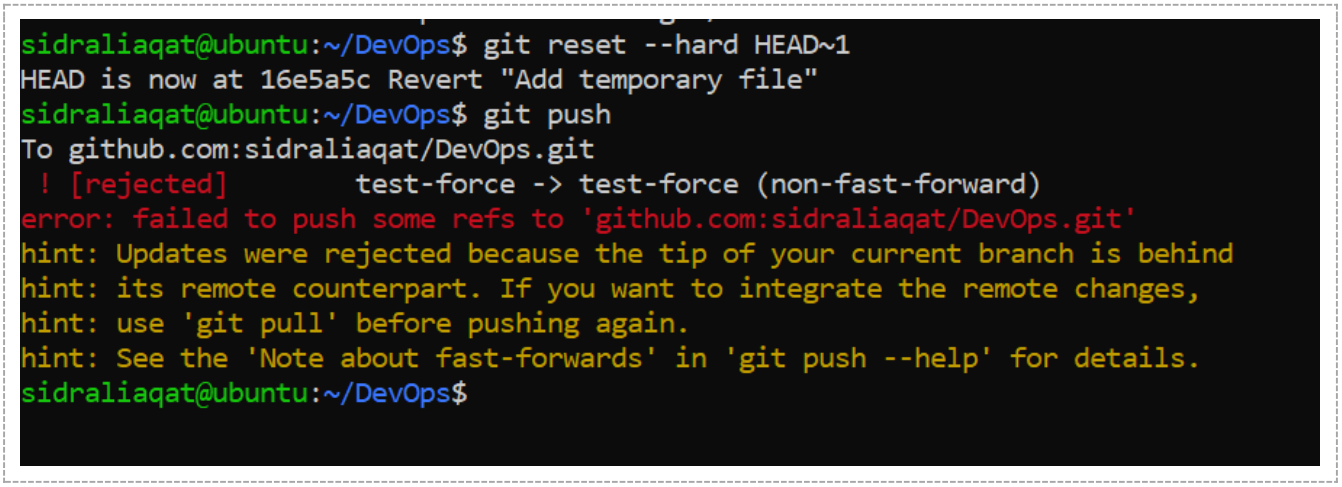
Screenshot: hard_reset_force.png



5. Push again. The remote rejects it.

```
git push origin test-force
```

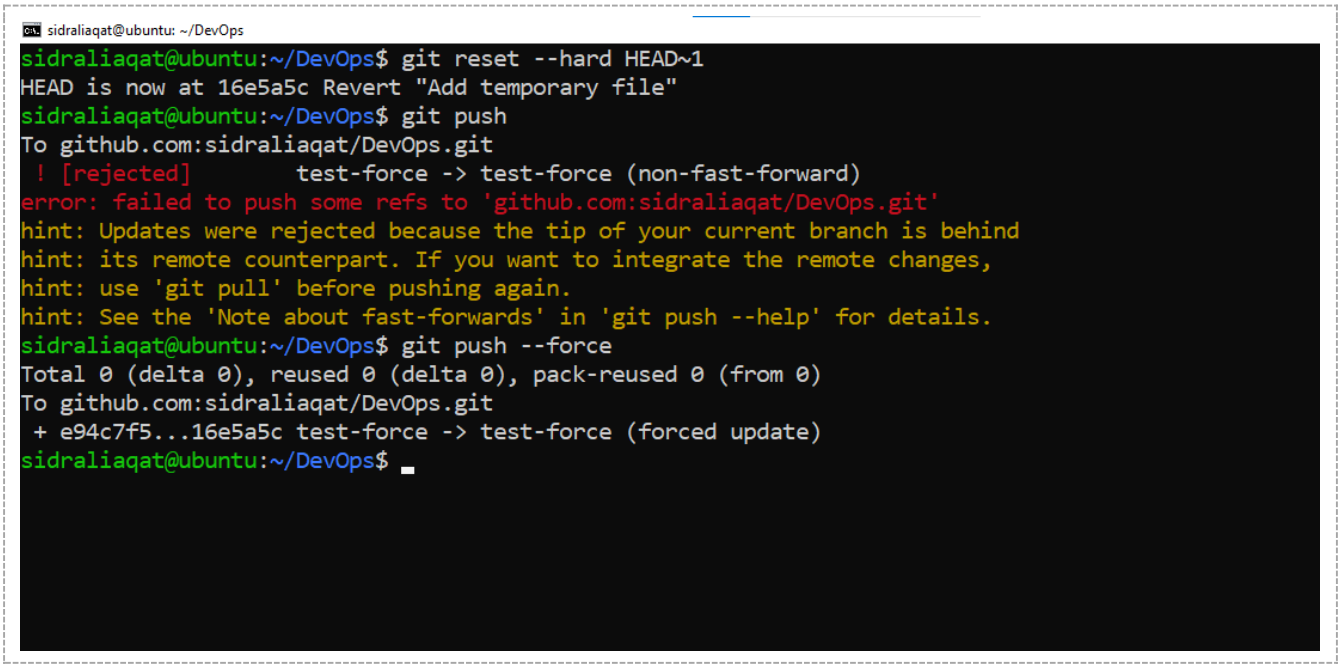
Screenshot: normal_push.png



6. Force push.

```
git push origin test-force --force
```

Screenshot: force_push.png

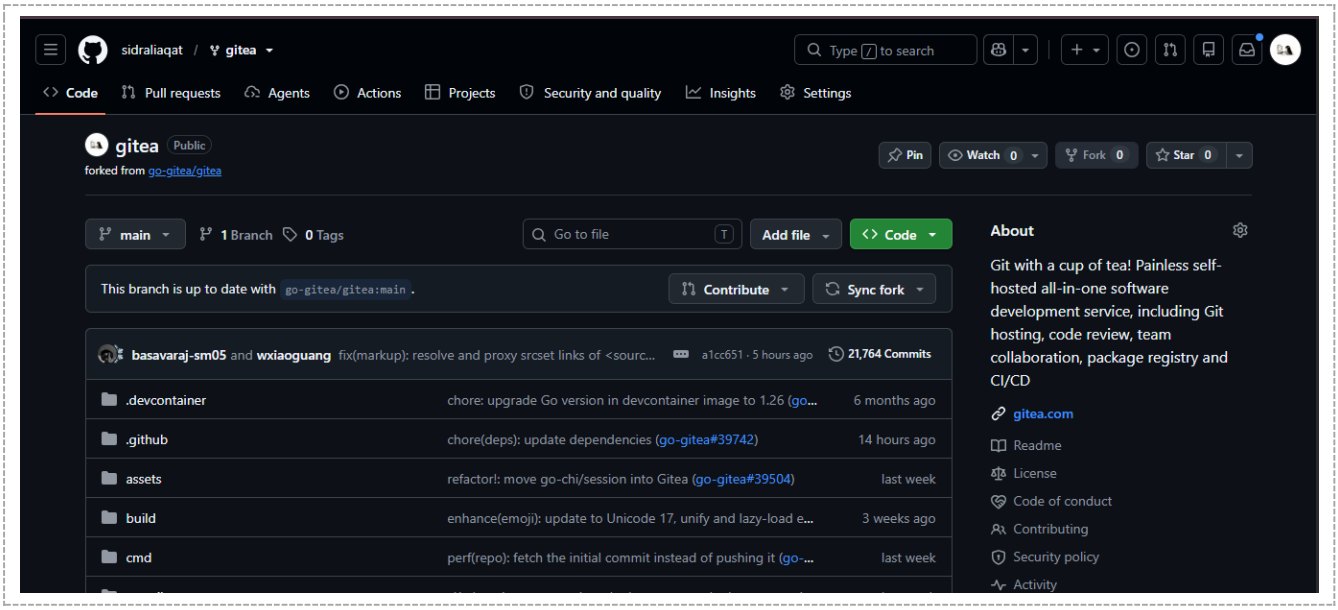


Task 10: Running Gitea in GitHub Codespaces via Docker Compose

Fork Gitea, run it in a Codespace with Docker Compose, install it, and create a repository inside it.

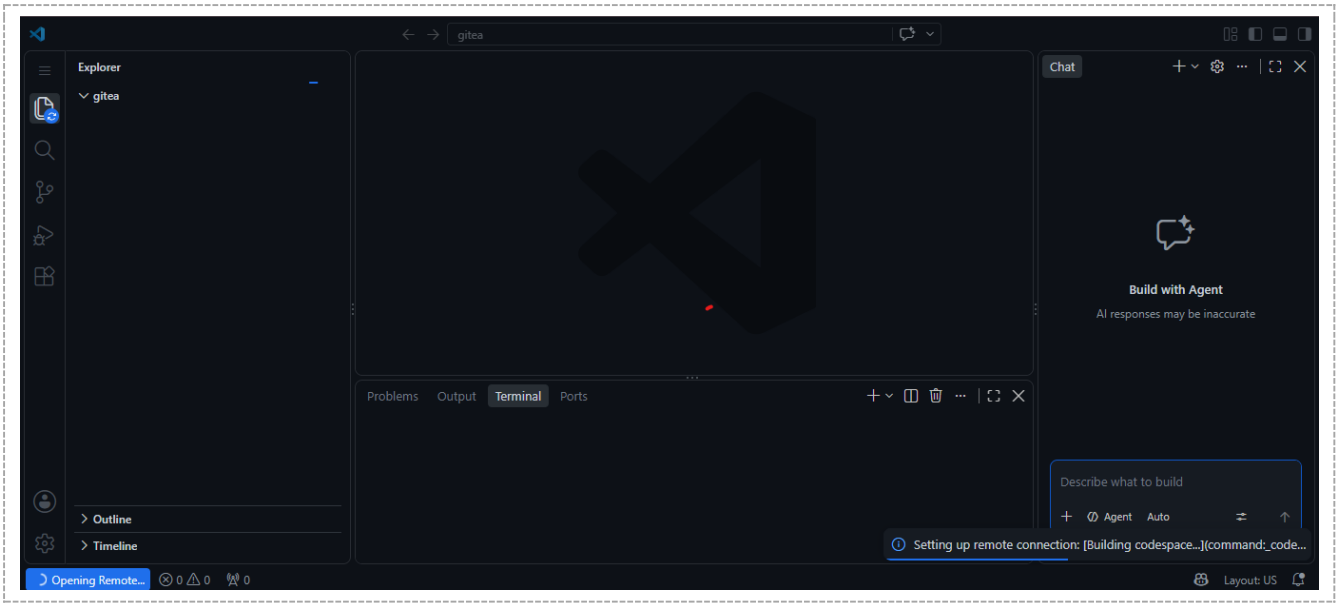
1. Fork the Gitea repository to your account. Uncheck "Copy the main branch only" so the codespace branch comes along.

   Screenshot: forked_gitea.png



2. On your fork, click Code > Codespaces > Create codespace on main.

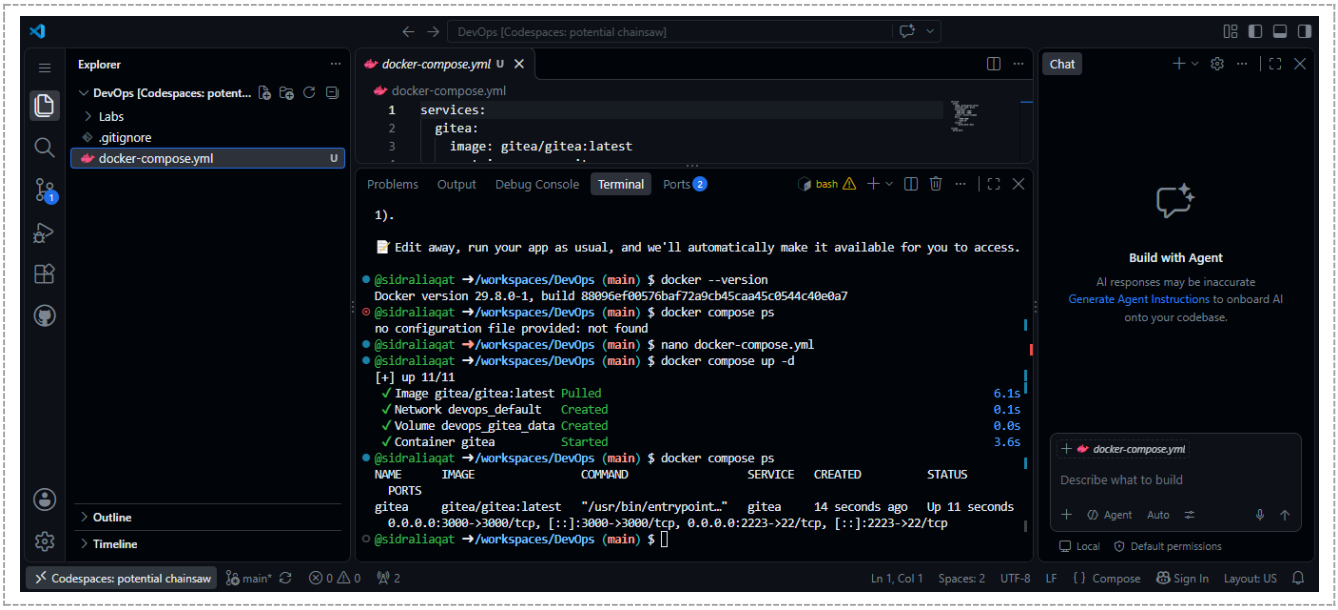
   Screenshot: codespace_loading.png



3. In the Codespace terminal, switch to the codespace branch and start Gitea.

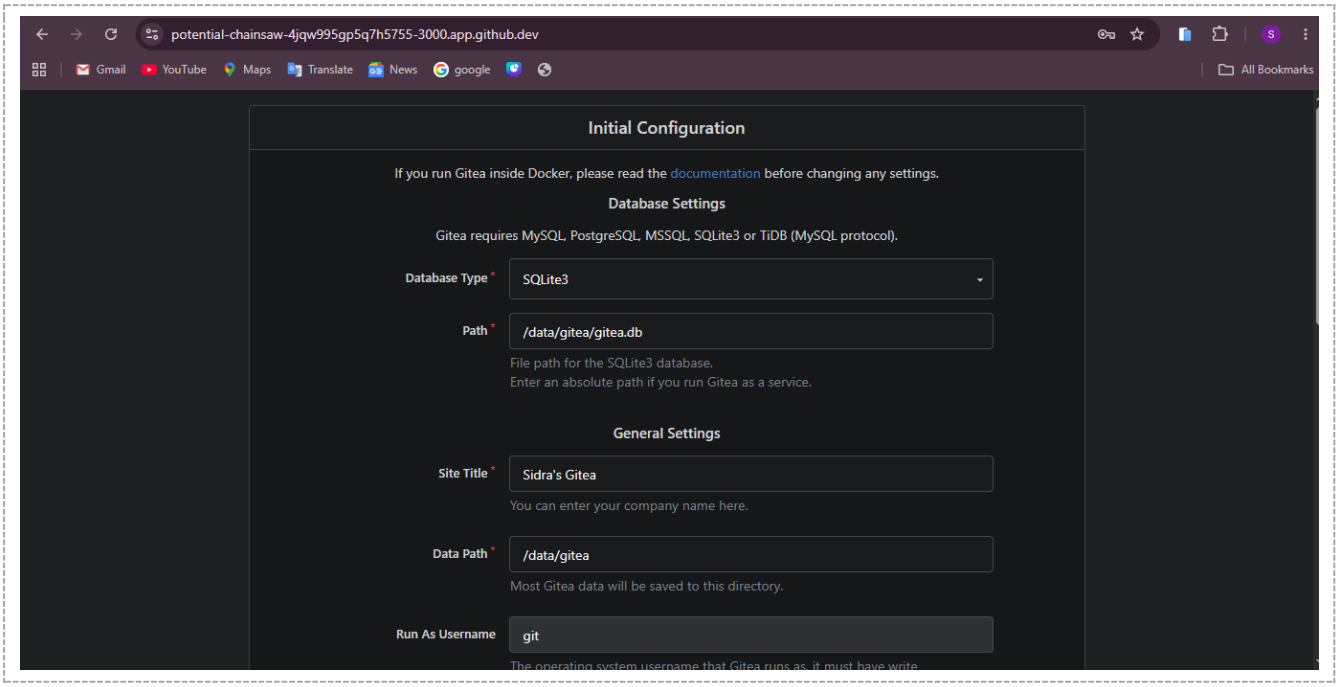
```
git checkout codespace
docker compose up -d
```

Screenshot: docker_up.png



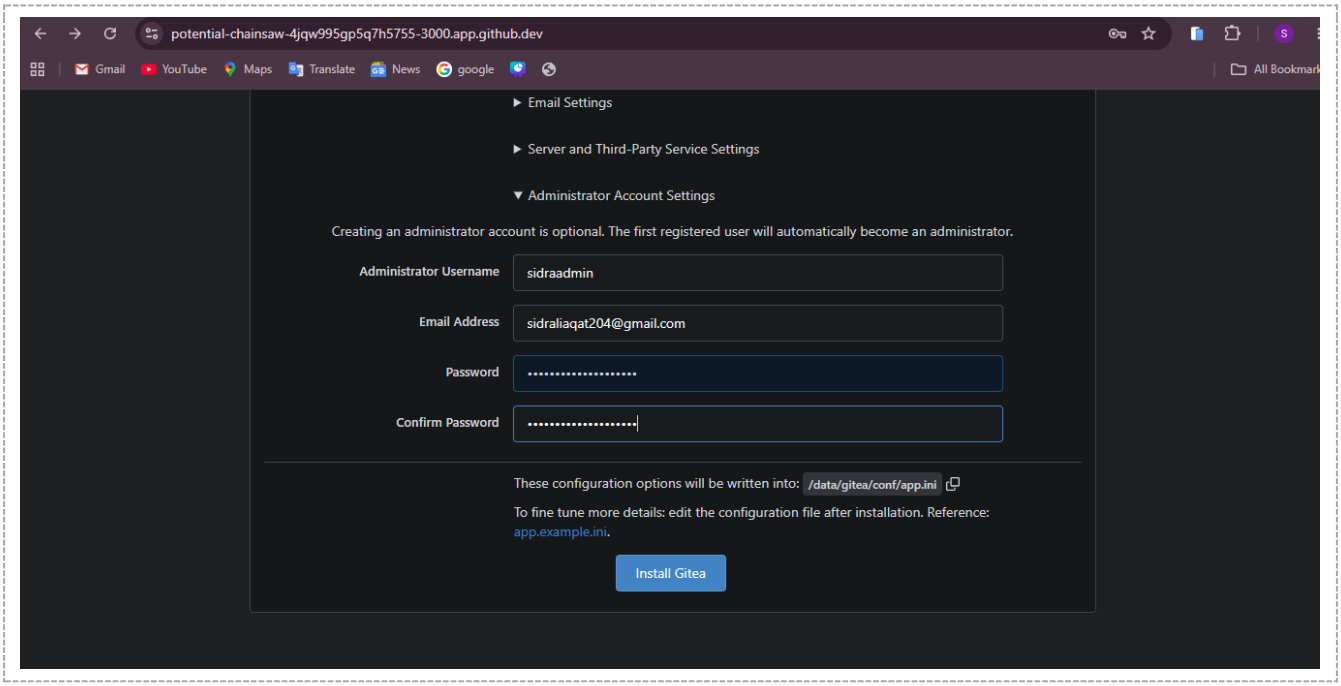
4. Forward port 3000, set its visibility to public, and open the link.

Screenshot: gitea_install_page.png



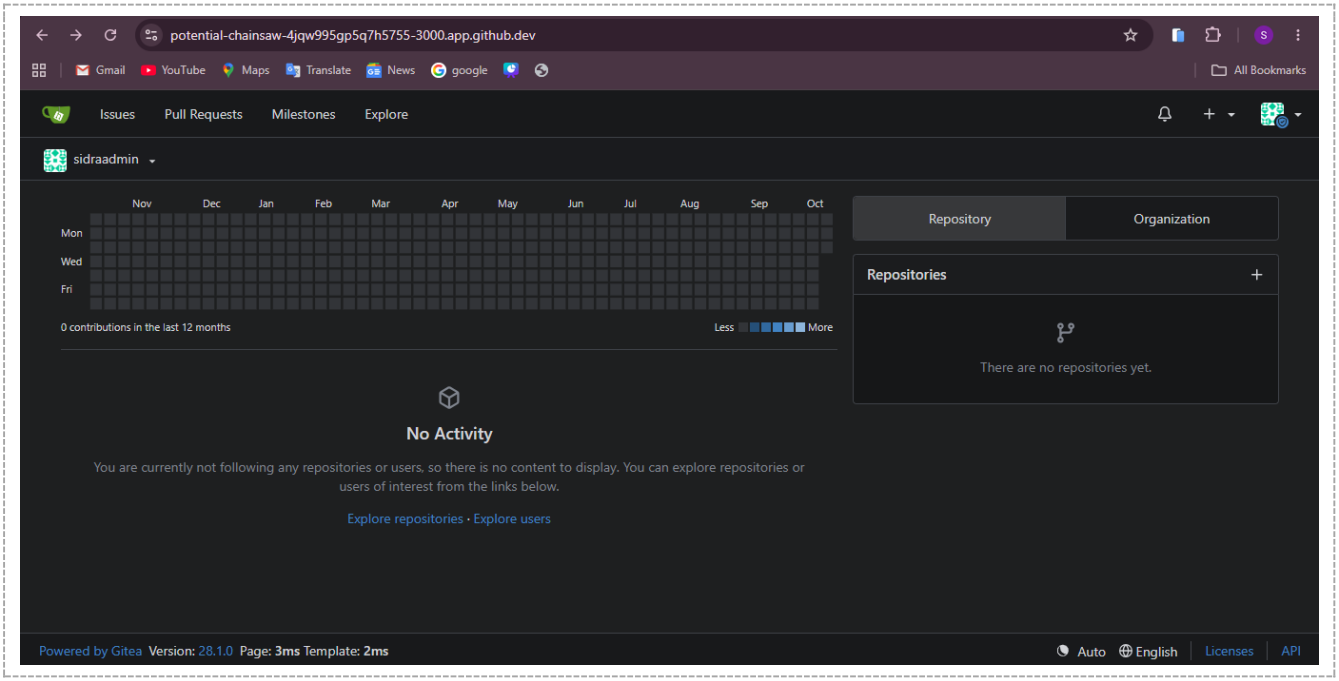
5. Fill in the install form with an admin username and password.

Screenshot: admin_setup.png



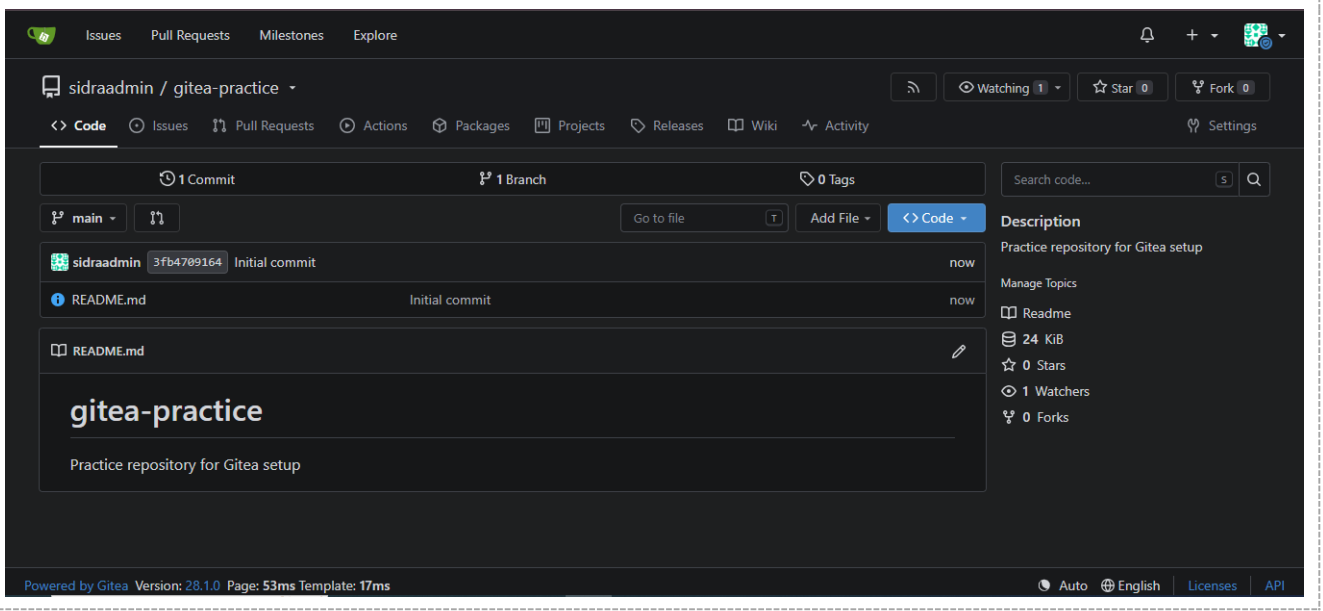
6. Log in with the admin account.

Screenshot: gitea_dashboard.png



7. Create a new repository in Gitea.

Screenshot: gitea_new_repo.png

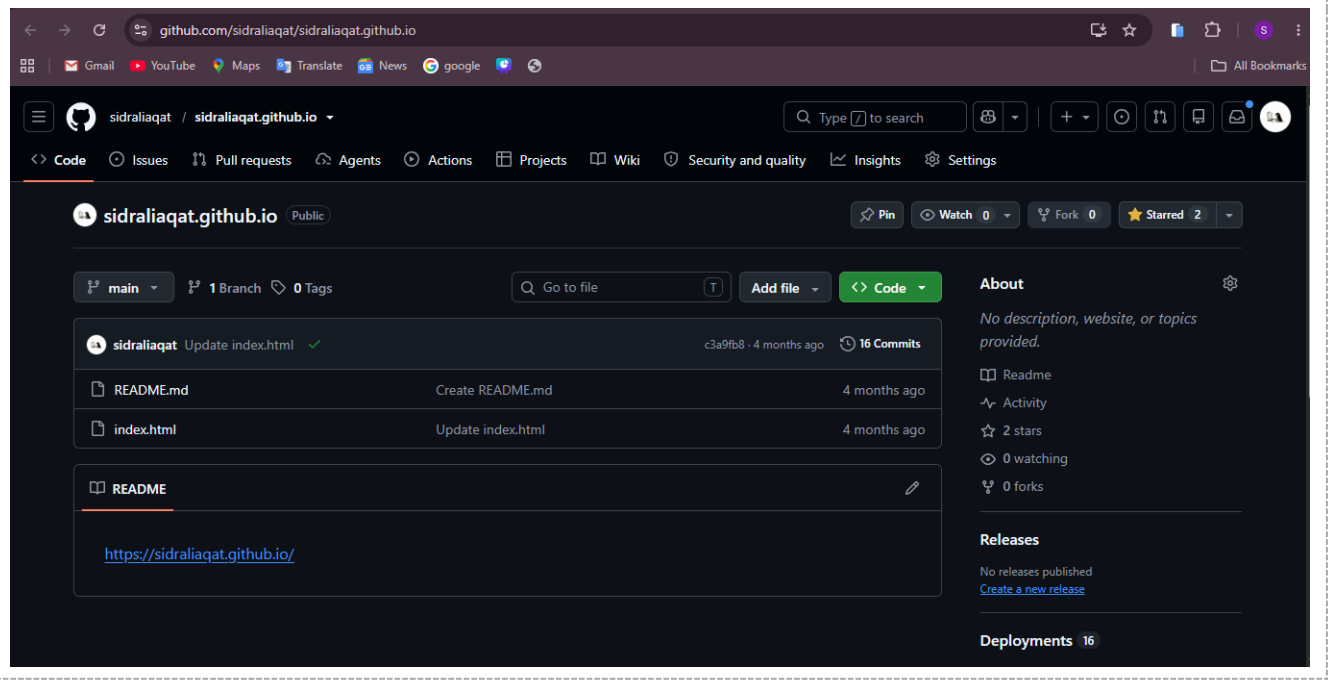


Task 11: Creating a GitHub Pages Portfolio Site

Publish a static portfolio or CV from a repository named <your-username>.github.io.

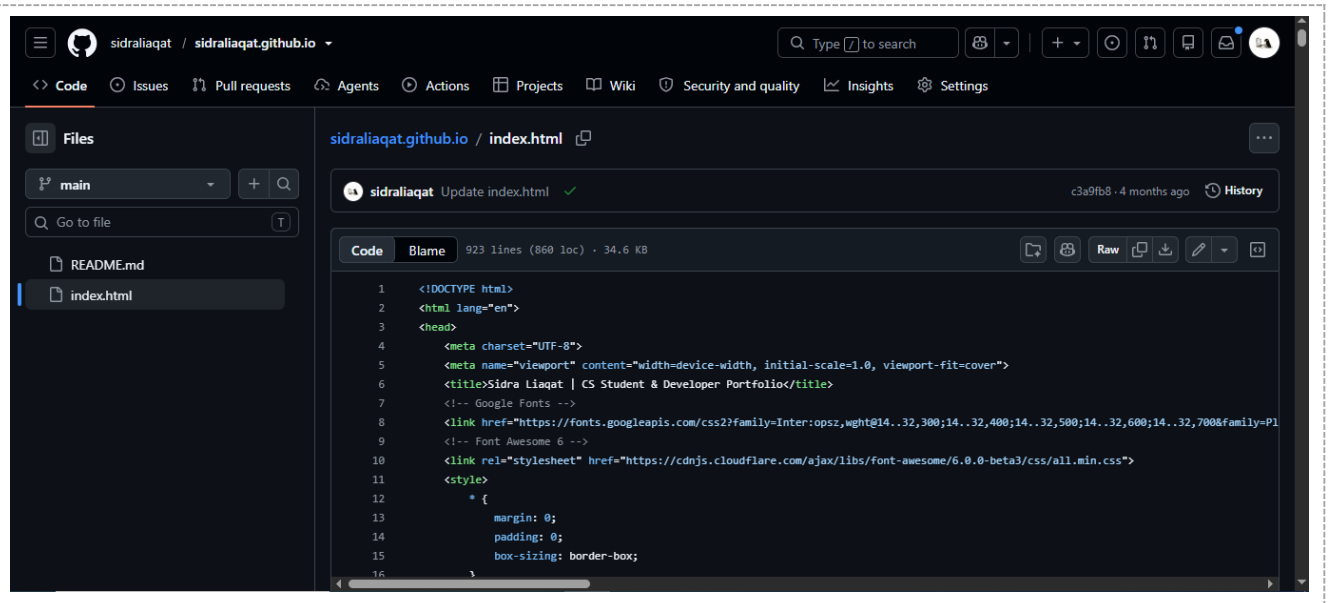
1. Create a public repository named <your-username>.github.io.

   Screenshot:



2. Put your static site files (HTML, CSS, JS) in a local folder.

   Screenshot: local_static_site.png

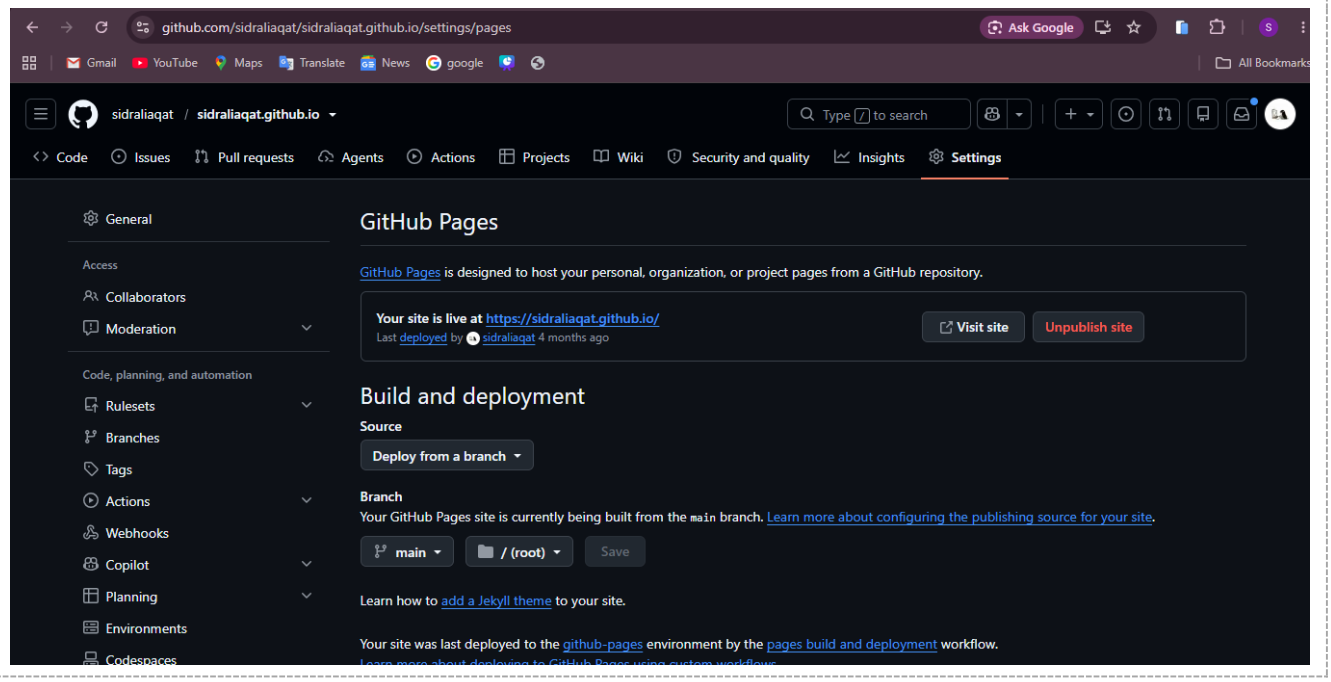


3. Initialize Git, then commit and push the files.

```
git init
git remote add origin git@github.com:<your-username>/<your-
username>.github.io.git
git add .
git commit -m "Add portfolio site for GitHub Pages"
git push -u origin main
```

4. Open Settings > Pages on the repository and confirm the site is published.

Screenshot: github_pages_settings.png



5. Open the live site in your browser.

Screenshot: live_site.png

